

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

HUỲNH TRẦN MINH KHANG
NGUYỄN DUY SƠN

NGHIÊN CỨU VÀ XÂY DỰNG THUẬT TOÁN SIMULATED
ANNEALING GIẢI BÀI TOÁN LẬP LỊCH JOB SHOP

ĐỒ ÁN CHUYÊN NGÀNH
NGÀNH: CÔNG NGHỆ THÔNG TIN

Thành phố Hồ Chí Minh, năm 2026

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

HUỲNH TRẦN MINH KHANG
NGUYỄN DUY SƠN

NGHIÊN CỨU VÀ XÂY DỰNG THUẬT TOÁN SIMULATED
ANNEALING GIẢI BÀI TOÁN LẬP LỊCH JOB SHOP

ĐỒ ÁN CHUYÊN NGÀNH
NGÀNH: CÔNG NGHỆ THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN:
TS. PHAN TẤN QUỐC

Thành phố Hồ Chí Minh, năm 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng báo cáo Đồ án chuyên ngành với đề tài " Nghiên cứu và xây dựng thuật toán Simulated Annealing giải bài toán lập lịch Job Shop" là công trình nghiên cứu độc lập của nhóm dưới sự hướng dẫn khoa học của TS. Phan Tấn Quốc.

Các kết quả thực nghiệm, dữ liệu phân tích và nội dung được trình bày trong báo cáo là trung thực, khách quan và được xây dựng dựa trên quá trình nghiên cứu lý thuyết cũng như cài đặt chương trình thực tế. Tất cả các nguồn tài liệu tham khảo, các thuật toán, mã nguồn hỗ trợ và dữ liệu chuẩn (benchmark) đều được trích dẫn và ghi nguồn rõ ràng theo đúng quy định học thuật.

Nhóm tác giả xin chịu hoàn toàn trách nhiệm về tính trung thực và các nội dung được trình bày trong báo cáo này.

Tác giả báo cáo

Huỳnh Trần Minh Khang

Nguyễn Duy Sơn

LỜI CẢM ƠN

Để hoàn thành đồ án này, bên cạnh sự nỗ lực của các thành viên trong nhóm, chúng em đã nhận được sự hỗ trợ và hướng dẫn vô cùng quý báu từ quý Thầy Cô và Nhà trường.

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới TS. Phan Tấn Quốc, người đã trực tiếp hướng dẫn và tận tình chỉ bảo chúng em trong suốt quá trình thực hiện đồ án chuyên ngành. Thầy đã dành nhiều thời gian định hướng đề tài, chia sẻ những kiến thức chuyên môn sâu sắc về tối ưu hóa cũng như truyền đạt những kinh nghiệm nghiên cứu khoa học quý báu. Những ý kiến đóng góp và sự khích lệ của Thầy là tiền đề quan trọng giúp chúng em vượt qua những thách thức về mặt giải thuật để hoàn thiện báo cáo một cách tốt nhất.

Chúng em cũng xin gửi lời cảm ơn đến Ban Giám hiệu Trường Đại học Sài Gòn cùng quý thầy cô Khoa Công nghệ Thông tin đã tạo điều kiện về cơ sở vật chất và môi trường học tập thuận lợi để chúng em có thể phát triển kỹ năng nghiên cứu và lập trình.

Cuối cùng, xin cảm ơn thầy cô và bạn bè đã luôn ủng hộ, khích lệ chúng em trong suốt thời gian qua. Dù đã có nhiều cố gắng, báo cáo khó tránh khỏi những hạn chế nhất định, nhóm chúng em rất mong nhận được những ý kiến đóng góp từ quý Thầy Cô để đề án ngày càng hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Tác giả báo cáo

Huỳnh Trần Minh Khang

Nguyễn Duy Sơn

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
MỤC LỤC	iii
DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT	vii
DANH MỤC CÁC BẢNG BIỂU	ix
DANH MỤC CÁC HÌNH ẢNH	x
MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN LẬP LỊCH JOB SHOP	4
1.1. Giới thiệu chi tiết vấn đề bài toán JSSP	4
1.1.1. Phát biểu bài toán	4
1.1.2. Mô hình toán học và các ràng buộc	5
1.1.2.1. Ràng buộc thứ tự công nghệ	5
1.1.2.2. Ràng buộc tài nguyên/máy	5
1.2. Các định lý và tính chất liên quan	5
1.2.1. Độ phức tạp NP-hard	5
1.2.2. Mô hình đồ thị rời rạc	6
1.2.3. Các loại lịch biểu	6
1.3. Khảo sát các hướng tiếp cận giải bài toán JSSP	7
1.3.1. Hướng tiếp cận trên thế giới	7
1.3.2. Hướng tiếp cận tại Việt Nam	8
1.4. Xác định vấn đề nghiên cứu của đề tài	9
1.5. Khảo sát hệ thống dữ liệu thực nghiệm chuẩn	9
1.5.1. Nguồn dữ liệu OR-Library	10
1.5.2. Các bộ dữ liệu trọng tâm trong nghiên cứu	10
1.6. Xác định tiêu chí đánh giá chất lượng thuật toán	10
1.7. Tóm tắt chương	11
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VỀ THUẬT TOÁN SIMULATED ANNEALING	13
2.1. Nguồn gốc vật lý và nguyên lý mô phỏng quá trình luyện kim	13
2.1.1. Bản chất của quá trình luyện kim nhiệt động lực học	13

2.1.2. Mối liên hệ giữa cơ học thống kê và thuật toán metropolis.....	14
2.1.3. Nguyên lý ánh xạ sang bài toán tối ưu hóa tổ hợp.....	14
2.2. Các thành phần và tham số cốt lõi của thuật toán SA.....	16
2.2.1. Không gian trạng thái và biểu diễn nghiệm.....	16
2.2.2. Hàm năng lượng và hàm mục tiêu.....	16
2.2.3. Tham số nhiệt độ.....	16
2.2.4. Cơ chế lân cận và các toán tử biến đổi.....	17
2.2.5. Chiều dài kỷ nguyên và chuỗi Markov.....	17
2.3. Cơ chế chấp nhận nghiệm và khả năng thoát tối ưu cục bộ.....	17
2.3.1. Thuật toán metropolis và tiêu chuẩn chấp nhận.....	18
2.3.2. Công thức xác suất metropolis.....	19
2.3.3. Chiến lược thoát tối ưu cục bộ.....	19
2.3.4. Sự hội tụ của thuật toán.....	20
2.4. Các chiến lược lập lịch làm lạnh và điều kiện dừng thuật toán.....	21
2.4.1. Khái niệm và vai trò của lịch làm lạnh.....	21
2.4.2. Các thông số cấu thành lịch làm lạnh.....	21
2.4.3. Các hàm giảm nhiệt phổ biến.....	21
2.4.4. Điều kiện dừng thuật toán.....	23
2.5. Ưu điểm và hạn chế của thuật toán SA khi so với thuật toán khác.....	24
2.5.1. So sánh về cơ chế vận hành.....	24
2.5.2. Ưu điểm của thuật toán SA.....	24
2.5.3. Hạn chế của thuật toán SA.....	24
2.6. Tóm tắt chương.....	25
CHƯƠNG 3. XÂY DỰNG THUẬT TOÁN SA GIẢI BÀI TOÁN LẬP LỊCH JOB SHOP	26
3.1. Đề xuất hướng tiếp cận bài toán JSSP bằng thuật toán SA cải tiến.....	26
3.1.1. Phân tích hạn chế của giải thuật SA truyền thống.....	26
3.1.2. Đề xuất cơ chế làm lạnh thích nghi.....	26
3.1.3. Đề xuất cơ chế chấp nhận nghiệm và kiểm soát sự hội tụ.....	27
3.2. Thiết kế giải thuật chi tiết.....	28
3.2.1. Phương pháp mã hóa lời giải dựa trên hoạt động.....	28
3.2.2. Xây dựng hàm mục tiêu và cách tính Makespan.....	28

3.2.3. Thiết kế các toán tử lân cận	29
3.3. Quy trình thực hiện thuật toán	29
3.3.1. Thiết lập các tham số đầu vào	30
3.3.2. Quy trình thực thi chi tiết	30
3.4. Thiết kế kiến trúc chương trình	32
3.4.1. Lựa chọn ngôn ngữ và thư viện hỗ trợ	32
3.4.2. Sơ đồ cấu trúc các module chức năng	32
3.4.3. Quy trình xử lý và đầu ra của hệ thống	33
3.5. Tóm tắt chương	33
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ	35
4.1. Chi tiết về bộ dữ liệu thực nghiệm Lawrence	35
4.1.1. Nguồn gốc và lịch sử	35
4.1.2. Quy mô và phân loại	35
4.1.3. Mục tiêu thực nghiệm	37
4.2. Giải mã cấu trúc file dữ liệu	38
4.2.1. Quy ước định dạng chung	38
4.2.2. Ví dụ minh họa thực tế	38
4.2.3. Chuyển đổi dữ liệu vào chương trình	39
4.3. Lựa chọn kịch bản thực nghiệm	39
4.3.1. Mục tiêu kịch bản	40
4.3.2. Danh sách các bài toán thực nghiệm chọn lọc	40
4.3.3. Quy trình thực hiện thực nghiệm	41
4.3.3.1. Số lần chạy và tính tái lập	41
4.3.3.2. Chỉ số ghi nhận và đánh giá	41
4.4. Thiết lập thông số và môi trường thực thi	42
4.4.1. Môi trường và dữ liệu	42
4.4.2. Thiết lập thông số thuật toán	43
4.5. Kết quả thực nghiệm và phân tích trực quan	44
4.5.1. Tổng hợp kết quả thực nghiệm	45
4.5.2. Phân tích chi tiết	45
4.5.3. Trực quan hóa và phân tích hành vi thuật toán	46

4.5.3.1. Phân tích biểu đồ hội tụ	46
4.5.3.2. Trực quan hóa lời giải	49
4.6. Đánh giá hiệu quả của các cải tiến.....	50
4.6.1. Cơ chế làm lạnh thích nghi	50
4.6.2. Cơ chế chấp nhận nghiệm và kiểm soát sự hội tụ.....	50
4.7. Tóm tắt chương.....	51
KẾT LUẬN VÀ KHUYẾN NGHỊ	52
TÀI LIỆU THAM KHẢO	54

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT

STT	Ký hiệu, chữ viết tắt	Ý nghĩa
1	JSSP	Job Shop Scheduling Problem (Bài toán lập lịch trong phân xưởng hay bài toán lập lịch Job Shop).
2	SA	Simulated Annealing (Thuật toán mô phỏng luyện kim).
3	NP-hard	Lớp các bài toán có độ phức tạp tính toán khó (không có giải thuật đa thức).
4	GA	Genetic Algorithm (Thuật toán di truyền).
5	OR-Library	Thư viện dữ liệu chuẩn cho các bài toán tối ưu hóa vận trù học.
6	BKS	Best Known Solution (Lời giải tốt nhất từng được công bố).
7	LA	Lawrence (Tên bộ dữ liệu Benchmark của JSSP được dùng trong thực nghiệm).
8	FT	Fisher and Thompson (Tên một bộ dữ liệu Benchmark của JSSP).
9	GUI	Graphical User Interface (Giao diện người dùng đồ họa).
10	Gap (%)	Độ lệch tương đối giữa nghiệm tìm được và cực trị (BKS).
11	SPT	Shortest Processing Time (Quy tắc ưu tiên công việc có thời gian xử lý ngắn nhất).
12	LPT	Longest Processing Time (Quy tắc ưu tiên công việc có thời gian xử lý dài nhất).
13	FIFO	First In First Out (Quy tắc Vào trước ra trước).

14	n	Tổng số lượng công việc (Jobs) trong bài toán.
15	m	Tổng số lượng máy (Machines) trong bài toán.
16	T_0	Nhiệt độ ban đầu của hệ thống.
17	T_{min} hoặc T_f	Nhiệt độ dừng hoặc Nhiệt độ kết thúc thuật toán.
18	α	Hệ số làm lạnh (tốc độ giảm nhiệt độ).
19	Δ	Độ chênh lệch giá trị hàm mục tiêu (năng lượng) giữa hai nghiệm.
20	L	Chiều dài chuỗi Markov (số vòng lặp tại mỗi mức nhiệt độ).
21	P	Xác suất chấp nhận một nghiệm tệ hơn theo tiêu chuẩn Metropolis.
22	C_{max}	Tổng thời gian hoàn thành công việc cuối cùng của hệ thống.
23	ε	Ngưỡng cải thiện (Phần trăm cải thiện tối thiểu để đánh giá sự thay đổi của thuật toán).
24	f_{reheat}	Hệ số hâm nóng (để tăng lại nhiệt độ khi thuật toán kích hoạt quá trình hâm nóng).

DANH MỤC CÁC BẢNG BIỂU

Bảng 2.1. Ánh xạ các thành phần của thuật toán SA vào bài toán JSSP	15
Bảng 4.1. Mô tả bộ dữ liệu Lawrence.....	36
Bảng 4.2. Danh sách các bài toán thực nghiệm	40
Bảng 4.3. Các thông số thuật toán	43
Bảng 4.4. Tổng hợp kết quả thực nghiệm	45

DANH MỤC CÁC HÌNH ẢNH

Hình 1.1. Mô hình đồ thị rời rạc	6
Hình 2.1. Sơ đồ nguyên lý ánh xạ	15
Hình 2.2. Cơ chế chấp nhận nghiệm Metropolis trong thuật toán SA	18
Hình 2.3. Sơ đồ chiến lược thoát tối ưu cục bộ	20
Hình 2.4. Quy trình làm lạnh hình học	22
Hình 2.5. Quy trình làm lạnh tuyến tính	23
Hình 4.1. Biểu đồ hội tụ của LA01	46
Hình 4.2. Biểu đồ hội tụ của LA20	47
Hình 4.3. Biểu đồ hội tụ của LA21	47
Hình 4.4. Biểu đồ hội tụ của LA36	48
Hình 4.5. Biểu đồ Gantt cho lời giải tốt nhất của LA20	49

MỞ ĐẦU

1. Lý do chọn đề tài

Trong nền sản xuất hiện đại, việc tối ưu hóa quy trình lập lịch sản xuất (Job Shop Scheduling Problem - JSSP) đóng vai trò then chốt trong việc giảm thiểu thời gian chờ, tiết kiệm chi phí và nâng cao năng suất. Tuy nhiên, JSSP là bài toán thuộc lớp NP-hard với không gian nghiệm khổng lồ, khiến các phương pháp truyền thống khó tìm được lời giải tối ưu trong thời gian ngắn. Thuật toán Simulated Annealing (SA) với khả năng thoát khỏi tối ưu cục bộ mạnh mẽ đã chứng minh được hiệu quả trong việc giải quyết các bài toán tối ưu tổ hợp phức tạp. Vì vậy, nhóm thực hiện đề tài này nhằm nghiên cứu và ứng dụng giải thuật SA để tìm lời giải tiệm cận tối ưu cho bài toán lập lịch, góp phần vào việc ứng dụng CNTT trong quản trị sản xuất.

2. Tổng quan nghiên cứu

Bài toán JSSP đã được nghiên cứu từ những năm 1950 với các quy tắc ưu tiên (Priority Rules). Sau đó, các phương pháp chính xác như quy hoạch động được áp dụng nhưng gặp hạn chế về quy mô bài toán. Những năm gần đây, xu hướng nghiên cứu chuyển dịch sang các thuật toán Metaheuristic như Genetic Algorithm (GA), Tabu Search và đặc biệt là Simulated Annealing. Tại Việt Nam, các công trình của các tác giả như Nguyễn Đức Nghĩa, Phan Quốc Khánh đã đặt nền móng lý thuyết vững chắc về tối ưu hóa rời rạc, tạo cơ sở để nhóm phát triển ứng dụng thực tiễn.

3. Mục đích và nhiệm vụ nghiên cứu

Mục đích: Nghiên cứu cơ sở lý thuyết và xây dựng chương trình ứng dụng thuật toán SA để giải quyết bài toán JSSP với mục tiêu tối thiểu hóa tổng thời gian hoàn thành (Makespan).

Nhiệm vụ:

- Hệ thống hóa lý thuyết về bài toán JSSP và giải thuật SA.

- Thiết kế phương pháp mã hóa lời giải và các toán tử lân cận phù hợp.
- Cài đặt thuật toán bằng ngôn ngữ Python.
- Thực nghiệm và đánh giá trên các bộ dữ liệu chuẩn (OR-Library).

4. Đối tượng và phạm vi nghiên cứu

Đối tượng: Bài toán lập lịch Job Shop và thuật toán Simulated Annealing.

Phạm vi: Tập trung vào bài toán JSSP tĩnh (số lượng công việc và máy cố định), không xét đến các yếu tố biến động như hỏng máy hay thời gian vận chuyển. Sử dụng ngôn ngữ Python để triển khai giải pháp.

5. Phương pháp nghiên cứu

Phương pháp nghiên cứu lý thuyết: Phân tích, tổng hợp tài liệu về tối ưu hóa và thuật toán metaheuristic.

Phương pháp thực nghiệm: Cài đặt phần mềm, chạy mô phỏng trên các bộ dữ liệu Benchmark quốc tế, so sánh kết quả và vẽ biểu đồ Gantt để kiểm chứng tính đúng đắn.

6. Giả thuyết khoa học

Nếu xây dựng được một cơ chế mã hóa lời giải hợp lý kết hợp với việc tinh chỉnh các thông số của thuật toán SA (nhiệt độ ban đầu, tốc độ làm lạnh) một cách khoa học, chương trình sẽ có khả năng tìm được các lịch trình có Makespan tiệm cận với kết quả tối ưu toàn cục trong một khoảng thời gian tính toán chấp nhận được.

7. Những đóng góp mới

Hệ thống hóa cách tiếp cận bài toán lập lịch Job Shop bằng thuật toán SA cho sinh viên ngành Công nghệ thông tin.

Xây dựng công cụ hỗ trợ lập lịch trực quan hóa qua biểu đồ Gantt, giúp người quản lý dễ dàng theo dõi kế hoạch sản xuất.

Đưa ra các phân tích thực nghiệm về ảnh hưởng của tham số nhiệt độ đến tốc độ hội tụ của thuật toán.

8. Cấu trúc của Đồ án chuyên ngành

Ngoài phần mở đầu, kết luận, danh mục tài liệu tham khảo và phụ lục, đồ án gồm có 04 chương:

- Chương 1: Tổng quan về bài toán lập lịch Job Shop: Trình bày khái niệm, phát biểu bài toán và phân tích độ phức tạp của JSSP.
- Chương 2: Cơ sở lý thuyết về thuật toán Simulated Annealing: Trình bày nguyên lý luyện kim, thuật toán Metropolis và các tham số cốt lõi của SA.
- Chương 3: Xây dựng thuật toán SA giải bài toán lập lịch Job Shop: Tập trung vào việc đề xuất hướng tiếp cận bài toán lập lịch Job Shop bằng thuật toán Simulated Annealing cải tiến.
- Chương 4: Thực nghiệm và đánh giá: Trình bày kết quả chạy trên các bộ dữ liệu LA01, LA16,... Hiển thị kết quả thực nghiệm, trực quan hóa và phân tích hiệu quả của thuật toán.

CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN LẬP LỊCH JOB SHOP

Chương 1 tập trung vào việc thiết lập cơ sở lý luận và thực tiễn cho đề tài nghiên cứu. Nội dung chương sẽ đi sâu vào việc định nghĩa bài toán lập lịch trong phân xưởng (JSSP) dưới góc độ toán học, phân tích các đặc tính kỹ thuật và độ phức tạp NP-hard của vấn đề. Đồng thời, chương này khảo sát các hướng tiếp cận giải quyết bài toán từ truyền thống đến hiện đại trên thế giới và tại Việt Nam. Cuối cùng, các hệ thống dữ liệu thực nghiệm chuẩn và tiêu chí đánh giá giải thuật sẽ được xác lập làm nền tảng để triển khai các thuật toán tối ưu trong những chương tiếp theo.

1.1. Giới thiệu chi tiết vấn đề bài toán JSSP

Bài toán lập lịch trong phân xưởng (Job Shop Scheduling Problem - JSSP) là một trong những bài toán tối ưu hóa tổ hợp kinh điển, đóng vai trò quan trọng trong việc điều hành sản xuất thực tế.

1.1.1. Phát biểu bài toán

Theo Michael L. Pinedo (2016) [7], bài toán JSSP được mô tả bởi một tập hợp n công việc (Jobs) được ký hiệu là $J = \{J_1, J_2, \dots, J_n\}$ cần được thực hiện trên một tập hợp m máy (Machines) $M = \{M_1, M_2, \dots, M_m\}$.

Mỗi công việc J_j bao gồm một chuỗi gồm n_j thao tác (Operations) $O_{1j}, O_{2j}, \dots, O_{m,j}$ phải được thực hiện theo một thứ tự công nghệ cố định. Mỗi thao tác O_{ij} yêu cầu xử lý trên một máy M_k cụ thể trong một khoảng thời gian không đổi p_{ij} và không được phép ngắt quãng trong quá trình thực hiện [7]. Mục tiêu cốt lõi của bài toán là tìm ra một phương án sắp xếp thứ tự thực hiện các thao tác trên các máy sao cho tổng thời gian hoàn thành công việc cuối cùng, gọi là **Makespan**(C_{max}), đạt giá trị tối thiểu.

1.1.2. Mô hình toán học và các ràng buộc

Dựa trên các nghiên cứu về tối ưu hóa của Nguyễn Đức Nghĩa [2] và Phan Quốc Khánh [1], mô hình toán học của JSSP phải tuân thủ nghiêm ngặt hai nhóm ràng buộc sau:

1.1.2.1. Ràng buộc thứ tự công nghệ

Đối với mọi công việc J_j , thao tác $O_{i,j}$ chỉ có thể bắt đầu sau khi thao tác $O_{i-1,j}$ đã hoàn thành.

$$S_{i,j} \geq S_{i-1,j} + p_{i-1,j}$$

Trong đó $S_{i,j}$ là thời điểm bắt đầu của thao tác i thuộc công việc j .

1.1.2.2. Ràng buộc tài nguyên/máy

Mỗi máy tại một thời điểm chỉ có thể xử lý tối đa một thao tác. Nếu hai thao tác $O_{i,j}$ và $O_{h,k}$ cùng yêu cầu xử lý trên một máy M_r , thì hoặc thao tác $O_{i,j}$ phải kết thúc trước khi $O_{h,k}$ bắt đầu, hoặc ngược lại. Điều này dẫn đến sự tranh chấp tài nguyên và là nguyên nhân chính gây ra sự bùng nổ tổ hợp trong tìm kiếm nghiệm [2], [5].

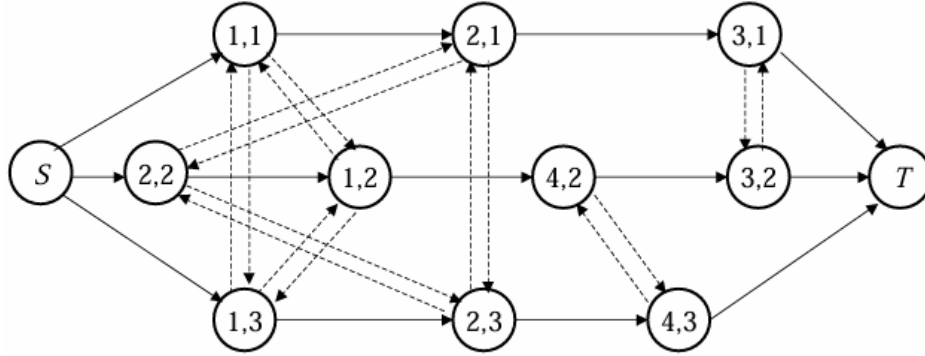
1.2. Các định lý và tính chất liên quan

1.2.1. Độ phức tạp NP-hard

Bài toán JSSP đã được chứng minh là thuộc lớp các bài toán NP-hard trong lý thuyết độ phức tạp tính toán [2]. Điều này có nghĩa là không tồn tại một thuật toán thời gian đa thức nào có thể tìm ra lời giải tối ưu cho mọi kích thước bài toán trong thời gian hữu hạn. Theo Nguyễn Hữu Mùi (2013) [3], sự phức tạp này bắt nguồn từ việc số lượng các phương án sắp xếp (lịch biểu khả thi) tăng theo hàm giai thừa khi số lượng công việc và máy tăng lên, khiến các phương pháp duyệt cạn trở nên bất khả thi.

1.2.2. Mô hình đồ thị rời rạc

Một công cụ quan trọng để phân tích tính chất của JSSP là mô hình đồ thị rời rạc $G = (V, C \cup D)$ [7]. Mô hình này giúp trực quan hóa các ràng buộc của bài toán:



Hình 1.1. Mô hình đồ thị rời rạc

Trong đó:

- **Tập đỉnh (V):** Đại diện cho tất cả các thao tác (Operations) của các công việc, bao gồm cả hai đỉnh giả định là đỉnh bắt đầu S và đỉnh kết thúc E .
- **Tập cung liên (C):** Biểu diễn thứ tự công nghệ bắt buộc bên trong mỗi công việc. Một cung từ $O_{i,j} \rightarrow O_{i+1,j}$ có trọng số là thời gian xử lý $p_{i,j}$.
- **Tập cung rời rạc (D):** Biểu diễn các cặp thao tác cùng yêu cầu một máy. Các cung này ban đầu là vô hướng (chưa xác định thứ tự).

1.2.3. Các loại lịch biểu

Trong lý thuyết lịch biểu, các nghiệm được phân loại để thu hẹp không gian tìm kiếm [2]:

- **Lịch biểu khả thi (Feasible Schedule):** Thỏa mãn tất cả các ràng buộc.
- **Lịch biểu tích cực (Active Schedule):** Không thể bắt đầu bất kỳ thao tác nào sớm hơn mà không làm chậm trễ ít nhất một thao tác khác.

- **Lịch biểu không trễ (Non-delay Schedule):** Không có máy nào bị bỏ trống nếu có ít nhất một thao tác đang chờ để được xử lý trên máy đó. Định lý quan trọng khẳng định rằng **nghiệm tối ưu của bài toán JSSP luôn nằm trong tập các lịch biểu tích cực** [2], [7]. Đây là cơ sở để các thuật toán như Simulated Annealing tập trung tìm kiếm hiệu quả hơn.

1.3. Khảo sát các hướng tiếp cận giải bài toán JSSP

Lịch sử nghiên cứu bài toán JSSP đã trải qua hơn nửa thế kỷ với nhiều bước tiến quan trọng. Các phương pháp giải quyết JSSP có thể được phân loại thành ba nhóm chính: Phương pháp chính xác, Phương pháp giải thuật kinh nghiệm (Heuristics) và Phương pháp siêu kinh nghiệm (Metaheuristics).

1.3.1. Hướng tiếp cận trên thế giới

Trên bình diện quốc tế, các nghiên cứu về JSSP phát triển rất mạnh mẽ, được hệ thống hóa rõ nét qua các công trình của Pinedo (2016) [7] và Talbi (2009) [8]:

- **Các phương pháp toán học chính xác:** Trong giai đoạn đầu (thập niên 1950-1960), các nhà khoa học tập trung vào các mô hình như Quy hoạch nguyên và giải thuật Nhánh và Cận. Theo Pinedo [7], các phương pháp này đảm bảo tìm ra lời giải tối ưu toàn cục. Tuy nhiên, do tính chất NP-hard của bài toán, thời gian tính toán tăng theo hàm mũ, khiến chúng chỉ khả thi với các bài toán quy mô rất nhỏ (ví dụ bài toán 10x10 kinh điển của Fisher và Thompson phải mất hơn 20 năm mới tìm được lời giải tối ưu chính xác).
- **Các quy tắc ưu tiên (Priority Rules):** Để đáp ứng yêu cầu tính toán nhanh trong thực tế, các quy tắc kinh nghiệm như SPT (Shortest Processing Time), LPT (Longest Processing Time), hay FIFO được áp dụng rộng rãi. Mặc dù các quy tắc này cực kỳ nhanh về tốc độ, nhưng

Talbi [8] chỉ ra rằng chất lượng nghiệm thường rất thấp và không ổn định khi cấu trúc bài toán thay đổi.

- **Sự bùng nổ của Metaheuristics:** Từ những năm 1980, một bước ngoặt lớn xảy ra khi Kirkpatrick và cộng sự (1983) [6] đề xuất giải thuật **Simulated Annealing (SA)**, dựa trên nguyên lý nhiệt động lực học. SA mở ra khả năng thoát khỏi "bẫy" tối ưu cục bộ bằng cơ chế chấp nhận các nghiệm tệ hơn với một xác suất nhất định [9]. Tiếp nối SA, các giải thuật như Di truyền (Genetic Algorithm), Tìm kiếm Tabu (Tabu Search) cũng được phát triển mạnh mẽ. Ưu điểm vượt trội của nhóm này là khả năng tìm thấy lời giải "đủ tốt" (tiệm cận tối ưu) trong thời gian "chấp nhận được" đối với các bài toán quy mô lớn [8].

1.3.2. Hướng tiếp cận tại Việt Nam

Tại Việt Nam, nghiên cứu về lý thuyết tối ưu hóa và lập lịch cũng nhận được sự quan tâm của nhiều học giả và đã đạt được những thành tựu đáng kể:

- **Nền tảng lý thuyết:** GS.TS. Nguyễn Đức Nghĩa [2] và GS.TS. Phan Quốc Khánh [1] là những người đặt nền móng vững chắc với các công trình về quy hoạch tuyến tính, quy hoạch rời rạc và vận trù học. Các giáo trình của các tác giả này cung cấp công cụ toán học để mô hình hóa các bài toán điều hành sản xuất phức tạp tại Việt Nam.
- **Nghiên cứu chuyên sâu về lịch biểu:** TS. Nguyễn Hữu Mùi (2013) [3] đã thực hiện các nghiên cứu hệ thống về thuật toán cho các bài toán lịch biểu, trong đó nhấn mạnh việc áp dụng các cấu trúc dữ liệu tiên tiến để tối ưu hóa quá trình tìm kiếm nghiệm.
- **Hướng tiếp cận cải tiến hiện đại:** Gần đây, TS. Lê Minh Tuấn (2022) [4] đã đề xuất nhiều hướng cải tiến cho các thuật toán lập lịch, đặc biệt là việc lai ghép (hybrid) giữa các giải thuật metaheuristic để tận dụng ưu điểm của từng loại. Các nghiên cứu này cho thấy xu hướng tại Việt

Nam đang tiệm cận rất gần với các nghiên cứu hiện đại trên thế giới, tập trung vào tính hiệu quả và khả năng ứng dụng thực tiễn của giải thuật [4], [5].

1.4. Xác định vấn đề nghiên cứu của đề tài

Dựa trên quá trình khảo sát, nhóm nhận thấy rằng mặc dù có nhiều phương pháp giải quyết JSSP, nhưng giải thuật **Simulated Annealing (SA)** vẫn là một lựa chọn tối ưu nhờ vào sự cân bằng giữa tính đơn giản trong cài đặt và hiệu quả trong việc tìm kiếm nghiệm toàn cục.

Vấn đề nghiên cứu trọng tâm của đề tài này được xác định rõ như sau:

1. **Mô hình hóa:** Chuyển đổi các ràng buộc vật lý của xưởng sản xuất sang mô hình toán học và cấu trúc dữ liệu mã hóa phù hợp cho giải thuật SA.
2. **Tối ưu hóa tham số:** Nghiên cứu ảnh hưởng của các tham số nhiệt độ ban đầu (T_0), hệ số làm nguội (α) và điều kiện dừng đến chất lượng nghiệm Makespan. Đây là thách thức lớn nhất vì mỗi bộ dữ liệu sẽ có những đặc điểm nhiệt độ tối ưu khác nhau [9].
3. **Thực thi giải thuật:** Sử dụng ngôn ngữ lập trình Python để cài đặt thuật toán, thực hiện việc tìm kiếm lân cận bằng các toán tử như hoán đổi hoặc đảo ngược để tìm kiếm trong không gian nghiệm khổng lồ của bài toán JSSP.
4. **Kiểm chứng:** Sử dụng dữ liệu thực nghiệm chuẩn từ quốc tế để khẳng định độ tin cậy của giải thuật.

1.5. Khảo sát hệ thống dữ liệu thực nghiệm chuẩn

Trong nghiên cứu về tối ưu hóa tổ hợp, việc sử dụng các bộ dữ liệu chuẩn (Benchmark) là yêu cầu bắt buộc để đảm bảo tính khách quan và khả năng so sánh giữa các thuật toán khác nhau. Đối với bài toán JSSP, hệ thống dữ liệu được thừa nhận rộng rãi nhất là **OR-Library**, một kho lưu trữ các bài toán kiểm thử được duy trì bởi cộng đồng tối ưu hóa quốc tế.

1.5.1. Nguồn dữ liệu OR-Library

OR-Library cung cấp các tập tin văn bản chứa thông tin cấu trúc của bài toán sản xuất, bao gồm số lượng công việc (n), số lượng máy (m), ma trận thứ tự máy ưu tiên và ma trận thời gian xử lý tương ứng. Đặc biệt, các dữ liệu này luôn đi kèm với giá trị **BKS (Best Known Solution)** – kết quả tốt nhất mà nhân loại từng tìm thấy, làm cột mốc để đánh giá hiệu quả của các thuật toán mới.

1.5.2. Các bộ dữ liệu trọng tâm trong nghiên cứu

Đề tài tập trung khảo sát các bộ dữ liệu có tính chất đại diện cao:

- **Bộ dữ liệu Fisher và Thompson (FT):** Gồm các bài toán FT06 (6×6), FT10 (10×10) và FT20 (20×5). Trong đó, FT10 được đánh giá là một trong những bài toán khó nhất lịch sử JSSP; dù chỉ có kích thước 10×10 nhưng giới khoa học đã mất hàng thập kỷ để tìm ra lời giải tối ưu là 930 [7].
- **Bộ dữ liệu Lawrence (LA):** Gồm 40 bài toán từ LA01 đến LA40 với nhiều quy mô khác nhau. Theo Talbi (2009) [8], các bộ dữ liệu này giúp đánh giá khả năng thích nghi của thuật toán khi không gian nghiệm thay đổi từ mức độ nhỏ (10×5) đến mức độ lớn (30×10).
- **Bộ dữ liệu Applegate và Cook (ORB):** Cung cấp các thử thách về tính hội tụ của thuật toán trong các kịch bản sản xuất phức tạp hơn.

1.6. Xác định tiêu chí đánh giá chất lượng thuật toán

Để đánh giá một thuật toán Metaheuristic như Simulated Annealing có hoạt động hiệu quả hay không, dựa trên các tiêu chí khoa học được hệ thống bởi Phan Quốc Khánh [1], Phạm Thế Quế [5] và Talbi [8]:

1. Chất lượng nghiệm (Makespan - C_{max}): Đây là tiêu chí quan trọng nhất.

Thuật toán phải tìm được giá trị C_{max} càng nhỏ càng tốt.

2. Độ lệch tương đối (Gap %): Tính toán khoảng cách phần trăm giữa nghiệm của thuật toán (C_{max}) và nghiệm tối ưu thế giới (BKS):

$$Gap(\%) = \frac{C_{max} - BKS}{BKS} \times 100$$

Trong đó:

- **C_{max} (Makespan):** Là kết quả tốt nhất mà chương trình Python tìm được.
- **BKS (Best Known Solution):** Là "kỷ lục thế giới" của bộ dữ liệu đó (ví dụ: FT10 là 930) được lấy từ OR-Library.
- **Ý nghĩa:** Gap càng gần 0% thì thuật toán càng tiệm cận sự hoàn hảo.

Theo Nguyễn Đức Nghĩa [2], một thuật toán được coi là tốt nếu RE (Relative Error – Sai số tương đối; Tương đương với Gap) duy trì ở mức thấp (thường là dưới 5%) trên các bộ dữ liệu chuẩn.

3. Thời gian tính toán: Đo lường thời gian CPU cần thiết để thuật toán kết thúc. Một thuật toán hiệu quả phải cân bằng được giữa chất lượng nghiệm và thời gian phản hồi, tránh tình trạng bùng nổ thời gian khi kích thước bài toán tăng.

4. Tính ổn định và độ tin cậy: Vì SA dựa trên các bước nhảy ngẫu nhiên, thuật toán cần được chạy nhiều lần (ví dụ 10 hoặc 30 lần) trên cùng một bộ dữ liệu để tính toán giá trị trung bình và độ lệch tương đối. Thuật toán có độ lệch tương đối càng thấp thì tính ổn định càng cao.

5. Tốc độ hội tụ: Khả năng nhanh chóng tìm thấy vùng không gian nghiệm tốt và tinh chỉnh nghiệm đó trong những giai đoạn cuối của quá trình làm lạnh.

1.7. Tóm tắt chương

Chương 1 đã thực hiện một cái nhìn toàn diện về bài toán JSSP. Thông qua việc phân tích mô hình toán học và các ràng buộc thực tế, nghiên cứu đã khẳng định tính

chất NP-hard của vấn đề, giải thích lý do tại sao các phương pháp chính xác gặp bế tắc khi quy mô bài toán tăng lên. Quá trình khảo sát các hướng tiếp cận tại Việt Nam và trên thế giới đã cho thấy sự chuyển dịch tất yếu sang các thuật toán siêu kinh nghiệm (Metaheuristics), trong đó SA nổi lên như một giải pháp mạnh mẽ nhờ cơ chế kiểm soát nhiệt độ thông minh để vượt qua tối ưu cục bộ. Cuối cùng, việc xác lập hệ thống dữ liệu OR-Library và các tiêu chí đánh giá nghiêm ngặt đã tạo ra một khung tham chiếu chuẩn mực. Đây chính là tiền đề then chốt để đi sâu vào thiết kế chi tiết giải thuật và đề xuất giải pháp kỹ thuật.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VỀ THUẬT TOÁN SIMULATED ANNEALING

Chương 2 tập trung hệ thống hóa cơ sở lý luận và cơ chế vận hành của thuật toán Tối ưu mô phỏng (Simulated Annealing - SA), một phương pháp metaheuristic lấy cảm hứng từ quá trình nhiệt luyện vật lý để giải quyết bài toán tối ưu hóa tổ hợp JSSP. Nội dung chương đi sâu phân tích từ nguồn gốc nhiệt động lực học, các tham số cốt lõi như nhiệt độ và năng lượng, đến cơ chế chấp nhận nghiệm Metropolis giúp thuật toán thoát khỏi bẫy tối ưu cục bộ. Đồng thời, chương cũng thảo luận về chiến lược lập lịch làm lạnh, điều kiện hội tụ và thực hiện đối sánh với các kỹ thuật khác nhằm khẳng định tính phù hợp và hiệu quả của SA đối với mục tiêu nghiên cứu của đề án.

2.1. Nguồn gốc vật lý và nguyên lý mô phỏng quá trình luyện kim

2.1.1. Bản chất của quá trình luyện kim nhiệt động lực học

Trong vật lý chất rắn, luyện kim là thuật ngữ mô tả quá trình nhiệt luyện một vật liệu để làm thay đổi các đặc tính lý hóa, giúp nó đạt tới trạng thái tinh thể ổn định nhất (trạng thái có mức năng lượng thấp nhất). Quá trình này diễn ra qua ba giai đoạn chính:

- **Giai đoạn nung nóng:** Vật liệu được nung lên tới nhiệt độ rất cao (T rất lớn), khiến các nguyên tử nhận được động năng lớn và bắt đầu dao động hỗn loạn. Ở trạng thái này, các cấu trúc tinh thể bị phá vỡ, vật chất chuyển sang trạng thái nóng chảy hoặc lỏng, nơi các nguyên tử có thể di chuyển tự do qua các cấu trúc sắp xếp khác nhau.
- **Giai đoạn hạ nhiệt có kiểm soát:** Nhiệt độ được giảm dần một cách cực kỳ chậm rãi. Tại mỗi mức nhiệt độ, hệ thống được giữ đủ lâu để các nguyên tử đạt tới trạng thái cân bằng nhiệt.
- **Giai đoạn kết tinh:** Khi nhiệt độ tiến dần về 0, động năng của các nguyên tử giảm đi. Do được hạ nhiệt chậm, các nguyên tử có đủ thời

gian để tự sắp xếp vào đúng vị trí trong mạng tinh thể, tạo ra một cấu trúc bền vững với mức năng lượng tối thiểu (ground state).

Ngược lại, nếu quá trình làm lạnh diễn ra quá nhanh (gọi là *quenching*), vật liệu sẽ bị "đóng băng" ở một trạng thái hỗn loạn, dẫn đến các khiếm khuyết trong mạng tinh thể, tương ứng với việc bị kẹt tại các tối ưu cục bộ trong toán học.

2.1.2. Mối liên hệ giữa cơ học thống kê và thuật toán metropolis

Nền tảng toán học của Simulated Annealing kế thừa từ thuật toán Metropolis trong lĩnh vực cơ học thống kê, điều này đã được Kirkpatrick và cộng sự hệ thống hóa lại để giải quyết các bài toán tối ưu tổ hợp.

Theo phân phối Boltzmann, xác suất để một hệ thống ở trạng thái cân bằng nhiệt tại nhiệt độ T có năng lượng E được tính theo công thức:

$$P(E) \propto \exp\left(-\frac{E}{k_B T}\right)$$

Trong đó k_B là hằng số Boltzmann.

Kirkpatrick và các cộng sự đã nhận ra rằng thuật toán Metropolis dùng để mô phỏng hành vi của các nguyên tử có thể được sử dụng như một công cụ tối ưu hóa mạnh mẽ. Điểm then chốt là cơ chế cho phép hệ thống chuyển từ trạng thái năng lượng thấp sang trạng thái năng lượng cao hơn với một xác suất nhất định phụ thuộc vào nhiệt độ (T), giúp hệ thống không bị kẹt ở các hố năng lượng địa phương [6].

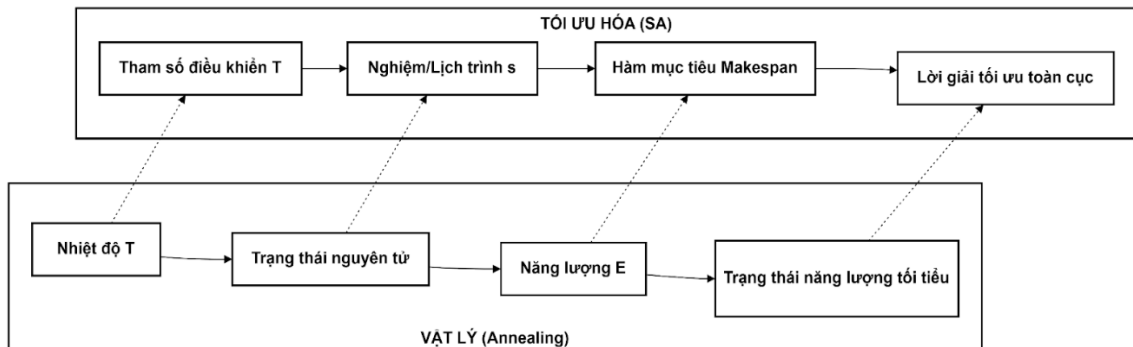
2.1.3. Nguyên lý ánh xạ sang bài toán tối ưu hóa tổ hợp

Để chuyển đổi từ một quá trình vật lý sang một thuật toán tính toán, SA thực hiện một sự ánh xạ trực tiếp giữa các đại lượng nhiệt động lực học và các thành phần của bài toán tối ưu hóa:

Bảng 2.1. Ảnh xạ các thành phần của thuật toán SA vào bài toán JSSP

Đại lượng trong vật lý (Annealing)	Thành phần trong Tối ưu hóa (SA)	Áp dụng vào bài toán JSSP
Trạng thái vật chất (s)	Lời giải khả thi (Nghiem)	Một thứ tự lập lịch cụ thể trên các máy.
Mức năng lượng (E)	Giá trị hàm mục tiêu (f(s))	Tổng thời gian hoàn thành (Makespan - $C_{\max}$).
Nhiệt độ (T)	Tham số điều khiển (T)	Biến số quyết định xác suất chấp nhận nghiệm tệ.
Sự di chuyển nguyên tử	Toán tử lân cận (Move)	Hoán đổi (Swap) hoặc chèn (Insert) các thao tác.
Trạng thái cân bằng	Sự hội tụ tại một mức nhiệt	Thực hiện đủ số lần lặp tại một nhiệt độ nhất định.
Trạng thái năng lượng tối thiểu	Lời giải tối ưu toàn cục	Lịch trình có Makespan ngắn nhất tìm được.

Nguyên lý này cho phép chúng ta xử lý JSSP — một bài toán vốn có không gian nghiệm cực kỳ lớn và nhiều "bẫy" tối ưu cục bộ — bằng cách "nung nóng" bài toán và cho nó "kết tinh" dần dần về phía lời giải tốt nhất [8].



Hình 2.1. Sơ đồ nguyên lý ánh xạ

2.2. Các thành phần và tham số cốt lõi của thuật toán SA

2.2.1. Không gian trạng thái và biểu diễn nghiệm

Không gian trạng thái S là tập hợp tất cả các phương án lập lịch khả thi thỏa mãn các ràng buộc về trình tự công nghệ của bài toán JSSP [3].

- **Trạng thái (s):** Trong nghiên cứu này, một trạng thái được biểu diễn dưới dạng một chuỗi hoán vị các thao tác. Đây là cách mã hóa phổ biến giúp đảm bảo mọi nghiệm được tạo ra luôn khả thi về mặt kỹ thuật.
- **Kích thước không gian:** Đối với bài toán có n công việc và m máy, số lượng trạng thái là cực kỳ lớn, khiến việc tìm kiếm vét cạn trở nên bất khả thi.

2.2.2. Hàm năng lượng và hàm mục tiêu

Trong SA, giá trị năng lượng E của hệ thống tương ứng trực tiếp với giá trị của hàm mục tiêu cần tối ưu hóa.

- **Định nghĩa:** Năng lượng của trạng thái s chính là tổng thời gian hoàn thành công việc (Makespan), ký hiệu là $f(s) = C_{max}$.
- **Công thức:** $C_{max} = \max_{C_{ij}}$, với C_{ij} là thời điểm hoàn thành của thao tác thuộc công việc i trên máy j .
- **Mục tiêu:** Thuật toán luôn hướng tới việc tìm kiếm các trạng thái có mức năng lượng thấp nhất (Makespan nhỏ nhất).

2.2.3. Tham số nhiệt độ

Nhiệt độ là tham số điều khiển quan trọng nhất, quyết định khả năng chấp nhận các nghiệm tệ hơn của thuật toán.

- **Nhiệt độ ban đầu (T_0):** Cần được thiết lập đủ cao để xác suất chấp nhận các nghiệm có $\Delta E > 0$ ở giai đoạn đầu xấp xỉ bằng 1. Điều này cho

phép thuật toán thực hiện quá trình thăm dò (Exploration) trên toàn bộ không gian nghiệm.

- **Nhiệt độ cuối (T_{min}):** Thường là một giá trị rất nhỏ gần bằng 0, tại đó thuật toán hầu như không chấp nhận thêm bất kỳ nghiệm tệ nào và bắt đầu hội tụ.

2.2.4. Cơ chế lân cận và các toán tử biến đổi

Để di chuyển từ trạng thái s sang trạng thái mới s' , thuật toán sử dụng các toán tử lân cận để tạo ra một sự thay đổi nhỏ trong cấu trúc lịch trình. Các toán tử dự kiến áp dụng bao gồm:

- **Toán tử Swap:** Hoán đổi vị trí của hai thao tác bất kỳ trong chuỗi hoán vị.
- **Toán tử Insert:** Lấy một thao tác và chèn vào một vị trí khác trong chuỗi.
- **Toán tử Invert:** Đảo ngược thứ tự của một đoạn thao tác trong chuỗi.

2.2.5. Chiều dài kỷ nguyên và chuỗi Markov

Chiều dài kỷ nguyên L (còn gọi là chiều dài chuỗi Markov) xác định số lần lặp lại (số lần thử nghiệm nghiệm mới) tại mỗi mức nhiệt độ nhất định [9].

- **Ý nghĩa:** Việc duy trì L đủ lớn giúp hệ thống đạt đến trạng thái "cân bằng nhiệt" tại mỗi mức nhiệt độ trước khi tiến hành giảm nhiệt.
- **Tác động:** Nếu L quá nhỏ, thuật toán sẽ hội tụ nhanh nhưng dễ bị kẹt vào tối ưu cục bộ; nếu L quá lớn, thời gian tính toán sẽ tăng đáng kể.

2.3. Cơ chế chấp nhận nghiệm và khả năng thoát tối ưu cục bộ

"Để thoát khỏi các tối ưu cục bộ, thuật toán đôi khi phải chấp nhận các bước đi làm tăng giá trị hàm mục tiêu" – đây là nguyên lý nền tảng của thuật toán Metropolis được ứng dụng trong giải thuật SA [6].

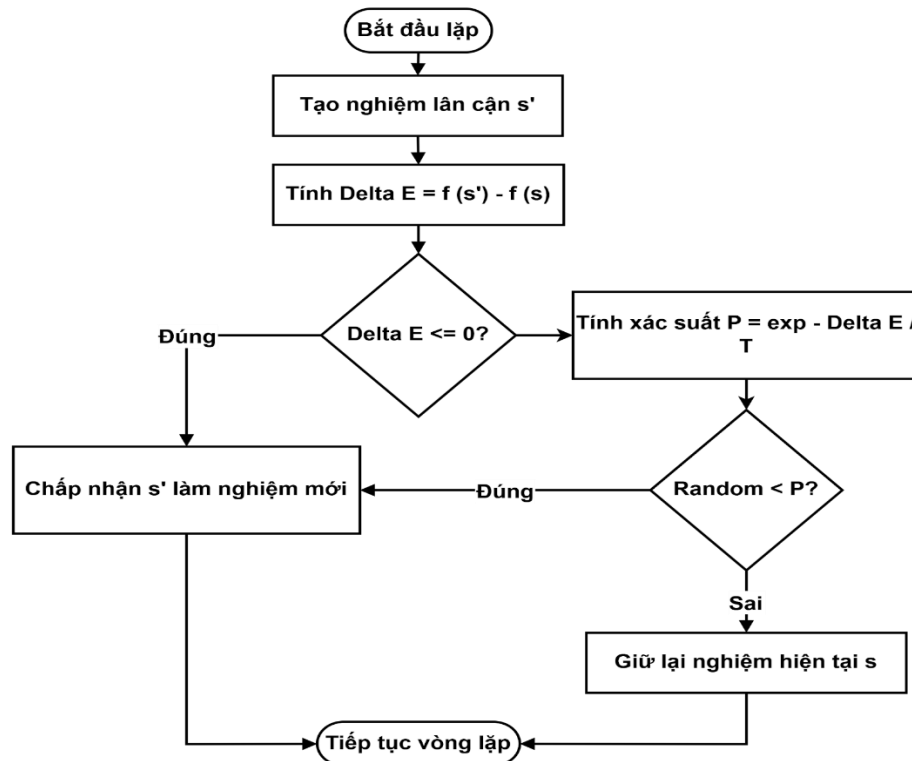
2.3.1. Thuật toán metropolis và tiêu chuẩn chấp nhận

Cơ chế cốt lõi giúp SA vượt qua các thuật toán tham lam là việc áp dụng tiêu chuẩn Metropolis. Trong mỗi bước lặp, thuật toán tạo ra một nghiệm lân cận s' từ nghiệm hiện tại s bằng các toán tử biến đổi. Sự thay đổi về "năng lượng" (giá trị Makespan trong bài toán JSSP) được xác định bởi:

$$\Delta E = f(s') - f(s)$$

Tiêu chuẩn chấp nhận nghiệm của Metropolis được thực hiện như sau:

- Nếu $\Delta E \leq 0$: Nghiệm mới tốt hơn hoặc bằng nghiệm hiện tại, thuật toán luôn luôn chấp nhận s' làm nghiệm mới cho bước tiếp theo.
- Nếu $\Delta E > 0$: Nghiệm mới tệ hơn nghiệm hiện tại, thuật toán không bác bỏ ngay lập tức mà thực hiện một phép thử ngẫu nhiên để quyết định có chấp nhận s' hay không.



Hình 2.2. Cơ chế chấp nhận nghiệm Metropolis trong thuật toán SA

2.3.2. Công thức xác suất metropolis

Xác suất chấp nhận một bước đi "tệ hơn" (làm tăng Makespan) được xác định bởi hàm số mũ Boltzmann:

$$P(\Delta E, T) = \exp\left(-\frac{\Delta E}{T}\right)$$

Trong đó:

ΔE : Độ chênh lệch giá trị hàm mục tiêu giữa nghiệm mới và nghiệm hiện tại.

T : Tham số nhiệt độ tại thời điểm tính toán.

Đặc tính của hàm xác suất P :

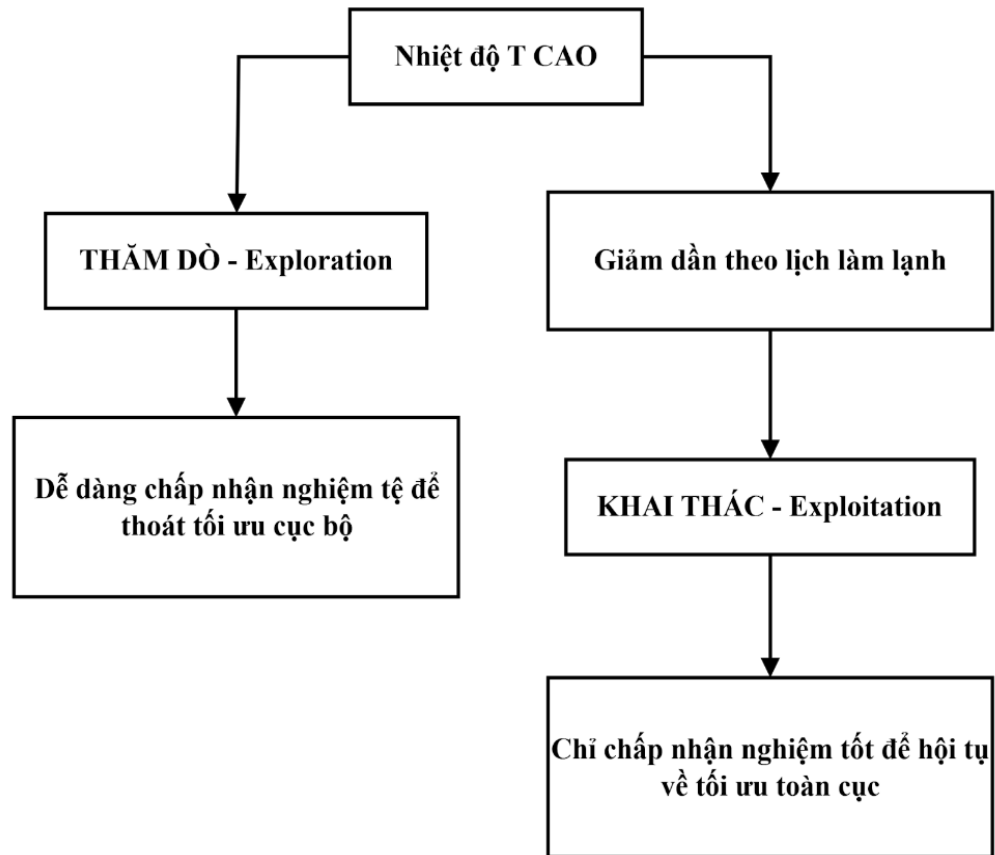
- Khi nhiệt độ T rất lớn (giai đoạn đầu): $P \approx 1$ nghĩa là hầu hết các nghiệm tệ đều có cơ hội được chấp nhận cao.
- Khi nhiệt độ T tiến về 0 (giai đoạn cuối): $P \approx 0$, thuật toán trở nên khắt khe và hầu như chỉ chấp nhận các nghiệm cải thiện.
- Với cùng một nhiệt độ T , nếu ΔE : càng lớn (nghiệm mới cực kỳ tệ), xác suất chấp nhận P sẽ càng thấp.

2.3.3. Chiến lược thoát tối ưu cục bộ

Khác với các thuật toán leo đồi (Hill Climbing) vốn chỉ chấp nhận các bước đi cải thiện và thường bị kẹt tại các "hố" tối ưu cục bộ, SA sử dụng xác suất Metropolis để thực hiện các bước nhảy "ngược dòng".

Giai đoạn thăm dò: Ở mức nhiệt độ cao, SA chấp nhận nhiều nghiệm tệ để di chuyển linh hoạt qua các vùng khác nhau của không gian nghiệm, giúp "nhảy" ra khỏi các cực trị địa phương.

Giai đoạn khai thác: Khi nhiệt độ giảm dần, xác suất chấp nhận nghiệm tệ giảm đi, thuật toán thu hẹp phạm vi và tập trung tinh chỉnh quanh vùng nghiệm tốt nhất đã tìm thấy để hội tụ về tối ưu toàn cục.



Hình 2.3. Sơ đồ chiến lược thoát tối ưu cục bộ

2.3.4. Sự hội tụ của thuật toán

Về mặt toán học, quá trình thực thi của SA có thể được mô tả như một chuỗi các chuỗi Markov. Nếu lịch làm lạnh được thiết lập đủ chậm, thuật toán được chứng minh là sẽ hội tụ tới tập hợp các nghiệm tối ưu toàn cục với xác suất bằng 1. Trong thực tế đồ án, chúng ta cần cân bằng giữa thời gian tính toán và chất lượng nghiệm bằng cách điều chỉnh hệ số làm lạnh α và chiều dài kỷ nguyên L phù hợp.

2.4. Các chiến lược lập lịch làm lạnh và điều kiện dừng thuật toán

2.4.1. Khái niệm và vai trò của lịch làm lạnh

Lịch làm lạnh quy định trình tự các giá trị nhiệt độ $\{T_0, T_1, \dots, T_k\}$ và số lượng lần lặp (chiều dài chuỗi Markov) tại mỗi mức nhiệt độ. Vai trò của nó là dẫn dắt hệ thống từ trạng thái hỗn loạn (nhiệt độ cao) đến trạng thái hội tụ ổn định (nhiệt độ thấp). Nếu nhiệt độ giảm quá nhanh, thuật toán sẽ bị kẹt tại tối ưu cục bộ; nếu giảm quá chậm, thời gian thực thi sẽ vượt quá mức cho phép [9].

2.4.2. Các thông số cấu thành lịch làm lạnh

Một lịch làm lạnh hoàn chỉnh thường sẽ bao gồm bốn thành phần chính sau đây:

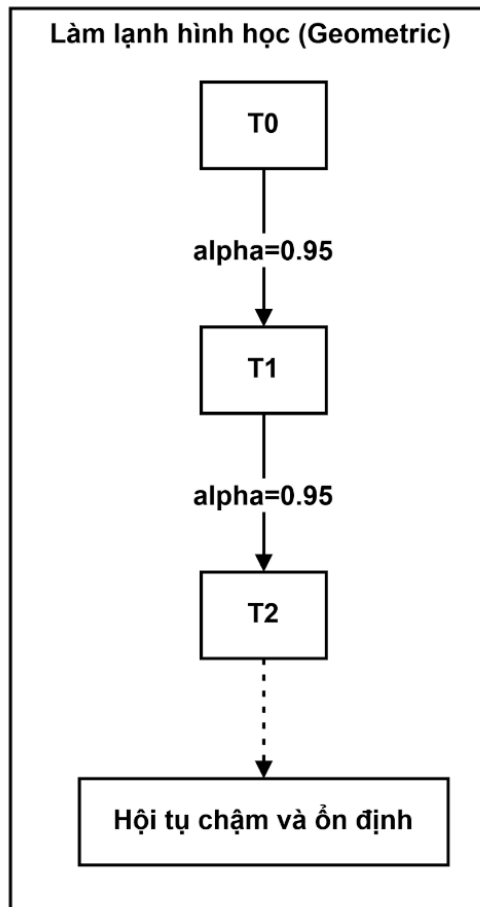
- **Nhiệt độ ban đầu (T_0):** Phải đủ lớn để tỷ lệ chấp nhận các nghiệm tệ lúc bắt đầu xấp xỉ 80% - 90%, giúp thuật toán thăm dò rộng khắp không gian nghiệm.
- **Hàm giảm nhiệt độ:** Xác định cách thức nhiệt độ T thay đổi sau mỗi kỷ nguyên (epoch).
- **Chiều dài chuỗi Markov (L)** Số lượng các bước di chuyển (moves) được thực hiện tại một mức nhiệt độ cố định trước khi tiến hành giảm nhiệt.
- **Nhiệt độ kết thúc (T_f):** Giá trị nhiệt độ đủ nhỏ để hệ thống được coi là đã "đóng băng"

2.4.3. Các hàm giảm nhiệt phổ biến

- Làm lạnh hình học (Geometric Cooling):

$$T_{k+1} = \alpha \times T_k \quad (0.8 \leq \alpha < 1)$$

Đây là phương pháp phổ biến nhất trong thực tế do tính đơn giản và khả năng hội tụ ổn định. Hệ số α thường được chọn rất gần 1 (ví dụ 0.95 hoặc 0.99) để đảm bảo quá trình hạ nhiệt diễn ra chậm rãi.

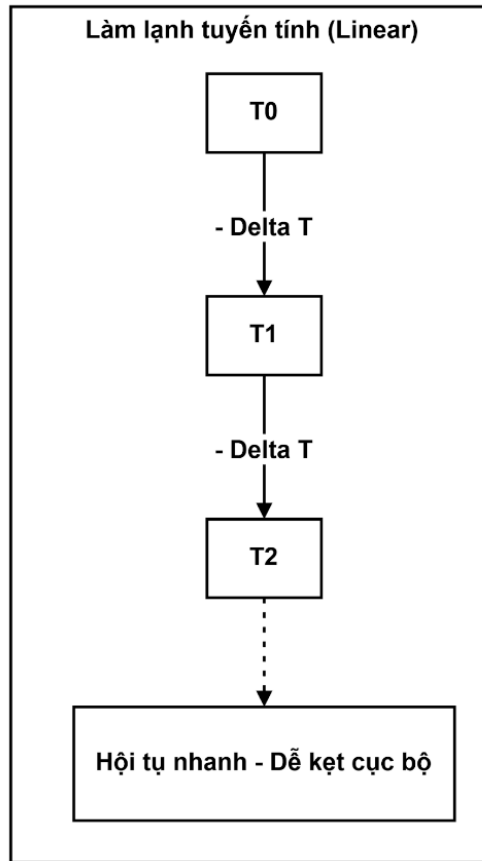


Hình 2.4. Quy trình làm lạnh hình học

- Làm lạnh tuyến tính (Linear Cooling):

$$T_{k+1} = T_k - \Delta T$$

Nhiệt độ giảm đi một lượng hằng số sau mỗi bước lặp. Phương pháp này dễ kiểm soát thời gian chạy nhưng có thể không phản ánh đúng bản chất cân bằng nhiệt của hệ thống ở các giai đoạn khác nhau.



Hình 2.5. Quy trình làm lạnh tuyến tính

2.4.4. Điều kiện dừng thuật toán

Điều kiện dừng xác định thời điểm thuật toán kết thúc quá trình tìm kiếm và trả về nghiệm tốt nhất tìm được (*Best_Known_Solution*) [9]. Các tiêu chí dừng phổ biến bao gồm:

- **Tiêu chí nhiệt độ:** Thuật toán dừng khi nhiệt độ hiện tại T thấp hơn ngưỡng T_f (ví dụ $T < 0.01$).
- **Tiêu chí ổn định:** Thuật toán dừng nếu sau một số lượng kỷ nguyên liên tiếp (ví dụ 50 hoặc 100 epoch), giá trị Makespan (C_{max}) không có sự cải thiện.
- **Tiêu chí thời gian/vòng lặp:** Dừng khi đạt đến tổng số lần lặp tối đa hoặc thời gian thực thi tối đa đã thiết lập.

2.5. Ưu điểm và hạn chế của thuật toán SA khi so với thuật toán khác

2.5.1. So sánh về cơ chế vận hành

Đối với Genetic Algorithm (GA): GA là giải thuật dựa trên quần thể, duy trì nhiều lời giải cùng lúc và sử dụng các toán tử lai ghép để tạo ra thế hệ mới. Trong khi đó, SA là giải thuật dựa trên một lời giải duy nhất, tập trung vào việc biến đổi và cải thiện một trạng thái tại mỗi thời điểm.

Đối với Tabu Search (TS): TS sử dụng cơ chế bộ nhớ (Tabu list) để ngăn chặn việc lặp lại các bước đi vừa thực hiện, từ đó ép buộc thuật toán khám phá các vùng không gian mới. Ngược lại, SA sử dụng cơ chế xác suất Metropolis để quyết định việc chấp nhận các nghiệm tệ hơn nhằm thoát khỏi tối ưu cục bộ.

2.5.2. Ưu điểm của thuật toán SA

Tính đơn giản và linh hoạt: SA có cấu trúc logic mạch lạc, dễ dàng cài đặt và triển khai trên các ngôn ngữ lập trình như Python.

Khả năng hội tụ: Về mặt lý thuyết, nếu lịch làm lạnh được thiết lập đủ chậm, SA được chứng minh là có khả năng hội tụ tới lời giải tối ưu toàn cục.

Ít tham số điều khiển: So với GA (đòi hỏi tinh chỉnh xác suất lai ghép, đột biến, kích thước quần thể), SA tập trung chủ yếu vào việc điều chỉnh nhiệt độ và tốc độ làm lạnh.

2.5.3. Hạn chế của thuật toán SA

Thời gian tính toán: Đối với các bài toán JSSP có quy mô cực lớn, SA có thể đòi hỏi thời gian thực thi dài để đạt được sự hội tụ cần thiết.

Độ nhạy tham số: Chất lượng lời giải cuối cùng phụ thuộc rất lớn vào việc lựa chọn nhiệt độ ban đầu (αT_0) và hệ số làm lạnh (α). Nếu giảm nhiệt quá nhanh, thuật toán dễ bị kẹt tại tối ưu cục bộ tương tự như các thuật toán tham lam.

"Sự thành công của Simulated Annealing không nằm ở chỗ nó luôn tìm ra lời giải tốt nhất, mà ở khả năng cung cấp một lời giải đủ tốt trong một khoảng thời gian chấp nhận được đối với các bài toán NP-hard." — Talbi, E. G. (2009)

2.6. Tóm tắt chương

Chương 2 đã hệ thống hóa toàn bộ nền tảng lý thuyết của thuật toán Simulated Annealing (SA), từ nguồn gốc nhiệt động lực học đến việc ánh xạ các tham số cốt lõi vào bài toán JSSP. Bằng cách làm rõ cơ chế chấp nhận nghiệm Metropolis – yếu tố then chốt giúp thoát khỏi các bẫy tối ưu cục bộ – cùng việc phân tích chiến lược làm lạnh và đối sánh với các kỹ thuật Metaheuristic khác, chương này đã khẳng định tính khả thi và hiệu quả của SA đối với mục tiêu nghiên cứu. Những cơ sở lý luận vững chắc này chính là tiền đề trực tiếp để triển khai thiết kế cấu trúc dữ liệu, mã hóa lời giải và cài đặt giải thuật chi tiết.

CHƯƠNG 3. XÂY DỰNG THUẬT TOÁN SA GIẢI BÀI TOÁN LẬP LỊCH JOB SHOP

Chương 3 tập trung vào việc hiện thực hóa giải thuật Simulated Annealing để giải quyết bài toán JSSP. Nội dung chương sẽ trình bày các đề xuất cải tiến về cơ chế làm lạnh, phương pháp mã hóa lời giải dựa trên hoạt động (Operation-based) và thiết kế hệ thống các toán tử lân cận. Bên cạnh đó, kiến trúc phần mềm trên ngôn ngữ Python cũng được xây dựng chi tiết, tạo tiền đề cho quá trình thực nghiệm và đánh giá.

3.1. Đề xuất hướng tiếp cận bài toán JSSP bằng thuật toán SA cải tiến

3.1.1. Phân tích hạn chế của giải thuật SA truyền thống

Giải thuật Simulated Annealing (SA) truyền thống khi áp dụng vào JSSP thường gặp phải một số thách thức lớn về hiệu năng và độ chính xác:

- **Tốc độ hội tụ:** Nếu hệ số làm lạnh α quá lớn, thuật toán dễ bị sa lầy vào tối ưu cục bộ. Ngược lại, nếu quá nhỏ, thời gian tính toán sẽ bùng nổ mà không mang lại hiệu quả tương xứng [8].
- **Sự nhạy cảm với tham số:** Theo Kirkpatrick (1983) [6], việc thiết lập nhiệt độ ban đầu T_0 quá cao sẽ khiến thuật toán mất nhiều thời gian ở giai đoạn tìm kiếm ngẫu nhiên, trong khi T_0 quá thấp sẽ làm mất đi khả năng vượt qua các "vách ngăn" năng lượng cao để tìm tới vùng nghiệm tốt hơn.
- **Không gian nghiệm JSSP:** Với tính chất NP-hard, không gian nghiệm của JSSP cực kỳ lồi lõm và phức tạp. SA truyền thống với một toán tử lân cận duy nhất thường không đủ linh hoạt để khám phá hết các góc ngách của không gian này [9].

3.1.2. Đề xuất cơ chế làm lạnh thích nghi

Để khắc phục các hạn chế trên và tạo ra sự đột phá cho đề tài, nhóm đề xuất sử dụng **Cơ chế làm lạnh thích nghi**. Thay vì sử dụng một hệ số α cố định

xuyên suốt quá trình, tốc độ giảm nhiệt sẽ được điều chỉnh dựa trên trạng thái hội tụ của nghiệm:

1. **Giai đoạn thăm dò:** Ở nhiệt độ cao, duy trì tốc độ làm lạnh chậm ($\alpha \approx 0.98$) để giải thuật tự do khám phá không gian nghiệm rộng lớn.
2. **Giai đoạn khai thác:** Giai đoạn khai thác: Khi Makespan ổn định (nhiệt độ $<$ ngưỡng), thuật toán chuyển sang $\alpha = 0.95$ (từ 0.98) để tăng tốc độ hạ nhiệt. Điều này giúp thu hẹp phạm vi tìm kiếm xung quanh vùng nghiệm tốt nhất và tập trung tinh chỉnh giải pháp cuối cùng.
3. **Công thức cập nhật nhiệt độ:**

$$T_{k+1} = \alpha \times T_k$$

Trong đó, α được cấu hình linh hoạt theo từng giai đoạn lặp (Markov chain) thay vì là một hằng số bất biến [9].

3.1.3. Đề xuất cơ chế chấp nhận nghiệm và kiểm soát sự hội tụ

Dựa trên thuật toán Metropolis [6], đề tài áp dụng cơ chế chấp nhận nghiệm mới S' từ nghiệm hiện tại S với xác suất P được xác định bởi công thức:

$$P(\text{accept } s') = \begin{cases} 1 & \text{nếu } f(s') < f(s) \\ \exp\left(-\frac{f(s') - f(s)}{T}\right) & \text{nếu } f(s') \geq f(s) \end{cases}$$

Trong đó:

- $f(s)$: Giá trị hàm mục tiêu của nghiệm hiện tại.
- $f(s')$: Giá trị hàm mục tiêu của nghiệm lân cận mới.
- T : Nhiệt độ hiện tại của hệ thống.

Điểm mới trong hướng tiếp cận của nhóm là thiết lập một "**ngưỡng dừng sớm**" (**Early Stopping**): Nếu sau một số lượng vòng lặp nhất định mà giá trị Makespan không được cải thiện quá một tỷ lệ ϵ , thuật toán sẽ tự động kết thúc

để tiết kiệm tài nguyên tính toán, hoặc thực hiện cơ chế "hâm nóng" (Reheating) để tái khám phá [8].

3.2. Thiết kế giải thuật chi tiết

3.2.1. Phương pháp mã hóa lời giải dựa trên hoạt động

Trong JSSP, việc chọn phương pháp mã hóa quyết định tính hiệu quả của giải thuật. Nhóm đề xuất sử dụng **Mã hóa dựa trên hoạt động (Operation-based Encoding)** vì nó luôn đảm bảo tạo ra các lịch trình khả thi mà không cần các bộ lọc sửa lỗi phức tạp [8].

- **Nguyên lý:** Một lời giải được biểu diễn bằng một hoán vị gồm $n \times m$ phần tử. Mỗi số hiệu công việc J_i sẽ xuất hiện đúng m lần trong dãy.
- **Cách đọc:** Lần xuất hiện thứ k của số i trong dãy sẽ đại diện cho thao tác thứ k của công việc J_i theo đúng thứ tự công nghệ.
- **Ví dụ:** Với bài toán 3 jobs, 2 machines (tổng 6 thao tác). Một lời giải có thể là: [2,1,2,3,1,3].
 - Số 2 đầu tiên: Thao tác 1 của Job 2.
 - Số 1 đầu tiên: Thao tác 1 của Job 1.
 - Số 2 thứ hai: Thao tác 2 của Job 2.
 - ... (cứ thế cho đến hết dãy).

3.2.2. Xây dựng hàm mục tiêu và cách tính Makespan

Hàm mục tiêu $f(S)$ của bài toán là giá trị Makespan. Để tính được giá trị này từ dãy mã hóa, chương trình cần quản lý hai mảng trạng thái:

1. `job_ready_time[j]`: Thời điểm sớm nhất mà Job j có thể bắt đầu thao tác kế tiếp.

2. `machine_ready_time[m]`: Thời điểm sớm nhất mà Máy m sẵn sàng để xử lý thao tác mới.

Công thức tính thời điểm hoàn thành (C) của thao tác $O_{i,j}$ trên máy k :

$$C(O_{i,j}) = \max(\text{job_ready_time}[j], \text{machine_ready_time}[k]) + p_{i,j}$$

Sau khi duyệt hết dãy mã hóa, giá trị Makespan được xác định là:

$$C_{max} = \max_{j \in J} (C_{last_operation,j})$$

3.2.3. Thiết kế các toán tử lân cận

Để tìm kiếm nghiệm trong không gian lân cận, thuật toán SA sẽ thực hiện các biến đổi nhỏ trên dãy mã hóa hiện tại. Nhóm đề xuất kết hợp hai toán tử để tăng tính đa dạng [8]:

- **Toán tử Hoán đổi:** Chọn ngẫu nhiên hai vị trí i và j trong dãy mã hóa và đổi chỗ chúng cho nhau. Toán tử này giúp thay đổi thứ tự ưu tiên giữa các công việc trên các máy một cách nhanh chóng.
- **Toán tử Dịch chuyển:** Chọn ngẫu nhiên một thao tác tại vị trí i và chèn nó vào một vị trí j bất kỳ trong dãy. Toán tử này tạo ra những thay đổi sâu hơn về cấu trúc lịch biểu so với Swap.

Chiến lược: Trong mỗi vòng lặp của SA, chương trình sẽ tung xúc xắc ngẫu nhiên để quyết định sử dụng Swap hay Move. Việc kết hợp này giúp giải thuật không bị lặp lại các cấu trúc nghiệm cũ và tăng khả năng tìm thấy vùng không gian nghiệm tốt hơn.

3.3. Quy trình thực hiện thuật toán

Để giải quyết bài toán JSSP, thuật toán Simulated Annealing được triển khai theo một quy trình tuần tự và chặt chẽ, mô phỏng quá trình luyện kim để tìm kiếm lời giải tối ưu. Quy trình này bao gồm việc thiết lập tham số đầu vào và thực thi các vòng lặp tìm kiếm lân cận.

3.3.1. Thiết lập các tham số đầu vào

Trước khi bắt đầu quá trình tối ưu hóa, các tham số điều khiển của thuật toán SA cần được xác định rõ ràng để đảm bảo tính hội tụ [6], [9]:

- **Nhiệt độ ban đầu (T_0):** Được thiết lập ở mức cao để cho phép thuật toán chấp nhận các nghiệm tệ hơn với xác suất lớn ở giai đoạn đầu, giúp thoát khỏi các vùng tối ưu cục bộ.
- **Nhiệt độ dừng (T_{min}):** Ngưỡng nhiệt độ mà tại đó thuật toán kết thúc (ví dụ: 0.01 hoặc 0.001).
- **Hệ số làm lạnh (α):** Tốc độ giảm nhiệt độ ($0 < \alpha < 1$), hệ số này sẽ thay đổi linh hoạt tùy theo giai đoạn và quyết định tốc độ giảm nhiệt sau mỗi vòng lặp.
- **Số vòng lặp tại mỗi mức nhiệt độ (L):** Còn gọi là chiều dài chuỗi Markov. Đây là số lượng phép thử lân cận được thực hiện tại mỗi mức nhiệt độ cố định để đảm bảo hệ thống đạt được trạng thái cân bằng nhiệt.
- **Dữ liệu đầu vào:** Ma trận thời gian xử lý và thứ tự máy từ các bộ dữ liệu Benchmark (FT10, LA, ...).

3.3.2. Quy trình thực thi chi tiết

Quy trình vận hành của giải thuật sẽ được chia thành các giai đoạn chính như sau:

Giai đoạn 1: Khởi tạo hệ thống

Thuật toán bắt đầu bằng việc sinh một lời giải ngẫu nhiên S_0 bằng phương pháp mã hóa Operation-based. Giá trị Makespan $f(S_0)$ được tính toán để làm mốc so sánh. Tại thời điểm này, S_0 cũng được coi là nghiệm tốt nhất hiện thời (S_{best}).

Giai đoạn 2: Vòng lặp tìm kiếm lân cận (Vòng lặp trong)

Tại mỗi mức nhiệt độ T hiện tại, thuật toán thực hiện L lần các thao tác sau:

1. Sử dụng toán tử lân cận (Swap hoặc Move) để biến đổi nghiệm hiện tại S thành một nghiệm mới S' .
2. Tính toán độ chênh lệch năng lượng (giá trị Makespan) giữa hai nghiệm: $\Delta = f(S') - f(S)$.
3. Xét điều kiện chấp nhận nghiệm:
 - Nếu $\Delta < 0$ (Nghiệm mới tốt hơn): Thuật toán chấp nhận ngay lập tức, gán $S = S'$. Nếu $f(S')$ tốt hơn cả $f(S_{best})$, tiến hành cập nhật $S_{best} = S'$.
 - Nếu $\Delta \geq 0$ (Nghiệm mới tệ hơn): Thuật toán tính xác suất chấp nhận $P = e^{-\Delta/T}$. Một số ngẫu nhiên r được sinh ra trong khoảng $[0,1]$. Nếu $r < P$, nghiệm tệ hơn vẫn được chấp nhận ($S = S'$) để duy trì khả năng tìm kiếm toàn cục. Nếu ngược lại, nghiệm S' bị loại bỏ.

Giai đoạn 3: Giảm nhiệt độ (Vòng lặp ngoài)

Sau khi hoàn thành L lần thử nghiệm ở mức nhiệt độ hiện tại, nhiệt độ được cập nhật giảm xuống theo công thức: $T = \alpha \times T$.

Giai đoạn 4: Điều kiện dừng và Xuất kết quả

Quy trình trên lặp lại liên tục cho đến khi nhiệt độ T giảm xuống dưới mức T_{min} . Lúc này, thuật toán dừng lại và xuất ra kết quả cuối cùng bao gồm: Giá trị Makespan nhỏ nhất tìm được và trình tự sắp xếp các công việc tối ưu trên các máy.

3.4. Thiết kế kiến trúc chương trình

3.4.1. Lựa chọn ngôn ngữ và thư viện hỗ trợ

Đề tài lựa chọn ngôn ngữ **Python** làm công cụ thực hiện nhờ vào sự linh hoạt và hệ sinh thái thư viện hỗ trợ tính toán mạnh mẽ:

- **NumPy**: Sử dụng để quản lý các ma trận thời gian xử lý và thứ tự máy dưới dạng mảng đa chiều, giúp tăng tốc độ tính toán hàm mục tiêu Makespan.
- **Matplotlib**: Thư viện chủ đạo để trực quan hóa kết quả. Nó được sử dụng để vẽ biểu đồ Gantt (lịch trình sản xuất) và đồ thị hội tụ của giá trị Makespan qua các thế hệ.
- **Thư viện Random & Math**: Cung cấp các hàm sinh số ngẫu nhiên cho các toán tử lân cận và các phép toán lũy thừa phục vụ công thức xác suất Metropolis.

3.4.2. Sơ đồ cấu trúc các module chức năng

Kiến trúc chương trình được chia thành 4 module chính với các nhiệm vụ riêng biệt:

1. Module Đọc dữ liệu (Data Loader):

- Nhiệm vụ: Đọc và phân tích các tệp dữ liệu chuẩn từ **OR-Library** (định dạng .txt).
- Đầu ra: Cung cấp ma trận thời gian (P) và ma trận máy (M) cho bộ giải.

2. Module Mô hình hóa (JSSP Model):

- Nhiệm vụ: Hiện thực hóa logic mã hóa Operation-based. Chứa hàm tính toán thời điểm bắt đầu/kết thúc của từng thao tác dựa trên các ràng buộc về tài nguyên máy và thứ tự công việc.

3. Module Giải thuật (SA Solver):

- Nhiệm vụ: "Trái tim" của chương trình. Thực hiện vòng lặp giảm nhiệt, quản lý các tham số T_0 , α , và triển khai các toán tử lân cận (Swap/Move).

4. Module Trực quan hóa (Visualizer):

- Nhiệm vụ: Chuyển đổi dữ liệu lập lịch từ dạng bảng số liệu sang dạng biểu đồ trực quan.

3.4.3. Quy trình xử lý và đầu ra của hệ thống

Hệ thống vận hành theo luồng dữ liệu khép kín:

- **Đầu vào (Input):** Người dùng cung cấp tệp dữ liệu Benchmark và các thông số cấu hình (T_0 , α , L).
- **Xử lý (Processing):** Module SA Solver phối hợp với JSSP Model để thực hiện hàng ngàn phép thử lân cận nhằm tìm ra dãy thứ tự thao tác tối ưu nhất.
- **Đầu ra (Output):**
 - Giá trị Makespan (C_{max}) tốt nhất tìm được.
 - **Biểu đồ Gantt:** Trực quan hóa tiến độ công việc trên từng máy, giúp người điều hành sản xuất dễ dàng theo dõi.
 - **Đồ thị hội tụ:** Biểu diễn quá trình giảm dần của Makespan theo thời gian, minh chứng cho sự hiệu quả của giải thuật.

3.5. Tóm tắt chương

Chương 3 đã hoàn thành việc xây dựng khung giải pháp cho đề tài thông qua việc đề xuất giải thuật Simulated Annealing cải tiến. Bằng cách áp dụng phương pháp mã hóa lời giải dựa trên hoạt động, nghiên cứu đã giải quyết triệt để các ràng buộc kỹ thuật của bài toán JSSP. Quy trình thực hiện thuật toán đã được mô tả chi tiết từ bước

khởi tạo đến cơ chế chấp nhận nghiệm Metropolis để vượt qua tối ưu cục bộ. Cuối cùng, một kiến trúc phần mềm Python bài bản đã được thiết kế, tạo nền tảng vững chắc cho giai đoạn thực nghiệm và đánh giá kết quả.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương 4 tập trung vào giai đoạn thực nghiệm nhằm kiểm chứng hiệu quả thực tế của thuật toán Simulated Annealing cải tiến trong việc giải quyết bài toán lập lịch Job Shop. Trong chương này, nhóm nghiên cứu sẽ đi sâu vào phân tích đặc điểm và cấu trúc của bộ dữ liệu Lawrence (LA) – một trong những thước đo tiêu chuẩn cho tính hiệu quả của các giải thuật tối ưu hóa. Thông qua các kịch bản thực nghiệm trên nhiều quy mô khác nhau, chương này sẽ trình bày các kết quả đạt được về giá trị Makespan, tốc độ hội tụ và biểu đồ Gantt trực quan, từ đó đưa ra những đánh giá khách quan về sức mạnh của cơ chế hâm nóng (Reheating) và làm lạnh thích nghi (Adaptive Cooling) mà nhóm đã đề xuất.

4.1. Chi tiết về bộ dữ liệu thực nghiệm Lawrence

Bộ dữ liệu Lawrence (thường được gọi tắt là bộ LA) là một trong những tập hợp các bài toán kiểm thử (benchmarks) quan trọng và phổ biến nhất trong nghiên cứu về bài toán lập lịch Job Shop.

4.1.1. Nguồn gốc và lịch sử

Bộ dữ liệu này được đề xuất bởi **S. Lawrence** vào năm 1984 và sau đó được tích hợp vào thư viện nghiên cứu vận trù học trực tuyến OR-Library của J.E. Beasley. Kể từ khi ra đời, bộ LA đã trở thành tiêu chuẩn vàng để đánh giá hiệu suất của các thuật toán tối ưu hóa tổ hợp như Simulated Annealing (SA), Genetic Algorithm (GA) hay Tabu Search.

4.1.2. Quy mô và phân loại

Bộ dữ liệu bao gồm 40 bài toán (từ la01 đến la40) với độ khó tăng dần dựa trên quy mô số lượng công việc (n) và số lượng máy (m). Các bài toán này được chia thành các nhóm chính như sau:

- **Nhóm quy mô nhỏ (la01 - la15):** Có số lượng máy cố định là 5, số lượng công việc thay đổi từ 10 đến 20. Đây là nhóm dữ liệu dùng để kiểm chứng tính đúng đắn của thuật toán.

- **Nhóm quy mô trung bình (la16 - la25):** Kích thước tăng lên 10×10 và 15×10 . Ở nhóm này, không gian tìm kiếm bắt đầu bùng nổ, đòi hỏi thuật toán phải có khả năng hội tụ tốt.
- **Nhóm quy mô lớn và phức tạp (la26 - la40):** Quy mô lên tới 30×10 và đặc biệt là nhóm 15×15 (la36 - la40). Đây là những bài toán cực kỳ khó (NP-hard), thường được dùng để thử thách các cơ chế cải tiến như làm lạnh thích nghi và hâm nóng nhiệt độ nhằm thoát khỏi tối ưu cục bộ.

Bảng 4.1. Mô tả bộ dữ liệu Lawrence

STT	Mã bộ dữ liệu	Số công việc	Số máy	BKS
1	LA01	10	5	666
2	LA02	10	5	655
3	LA03	10	5	597
4	LA04	10	5	590
5	LA05	10	5	593
6	LA06	15	5	926
7	LA07	15	5	890
8	LA08	15	5	863
9	LA09	15	5	951
10	LA10	15	5	958
11	LA11	20	5	1222
12	LA12	20	5	1039
13	LA13	20	5	1150
14	LA14	20	5	1292
15	LA15	20	5	1207
16	LA16	10	10	945
17	LA17	10	10	784
18	LA18	10	10	848
19	LA19	10	10	842

STT	Mã bộ dữ liệu	Số công việc	Số máy	BKS
20	LA20	10	10	902
21	LA21	15	10	1046
22	LA22	15	10	927
23	LA23	15	10	1032
24	LA24	15	10	935
25	LA25	15	10	977
26	LA26	20	10	1218
27	LA27	20	10	1235
28	LA28	20	10	1216
29	LA29	20	10	1152
30	LA30	20	10	1355
31	LA31	30	10	1784
32	LA32	30	10	1850
33	LA33	30	10	1719
34	LA34	30	10	1721
35	LA35	30	10	1888
36	LA36	15	15	1268
37	LA37	15	15	1397
38	LA38	15	15	1196
39	LA39	15	15	1233
40	LA40	15	15	1222

4.1.3. Mục tiêu thực nghiệm

Trong đồ án này, bộ dữ liệu Lawrence đóng vai trò là "thước đo" để nhóm thực hiện các mục tiêu sau:

1. **Kiểm chứng độ chính xác:** So sánh giá trị Makespan (C_{max}) tìm được với giá trị tối ưu đã biết (Best Known Solution - BKS) để tính toán độ lệch tương đối (Gap%).

2. **Đánh giá sự ổn định:** Kiểm tra khả năng tìm được nghiệm tốt của thuật toán qua nhiều lần chạy khác nhau trên cùng một bộ dữ liệu.
3. **Phân tích tốc độ:** Đo lường thời gian xử lý khi quy mô bài toán tăng dần từ 10×5 lên 15×15 .

Việc sử dụng một bộ dữ liệu chuẩn quốc tế như Lawrence giúp kết quả thực nghiệm của đề án có tính khách quan và có khả năng đối soát trực tiếp với các nghiên cứu khoa học khác trong cùng lĩnh vực.

4.2. Giải mã cấu trúc file dữ liệu

Để thuật toán Simulated Annealing có thể xử lý và tối ưu hóa, việc phân tích và chuẩn hóa dữ liệu đầu vào là bước tiên quyết. Các tập tin trong bộ dữ liệu Lawrence (LA) được lưu trữ dưới dạng văn bản thuần túy (.txt) với quy ước định dạng chặt chẽ để biểu diễn các ràng buộc về máy và thời gian.

4.2.1. Quy ước định dạng chung

Cấu trúc của một file la*.txt được chia thành hai phần chính:

- **Dòng tiêu đề (Header):** Chứa hai số nguyên n và m . Trong đó, n là tổng số công việc (Jobs) và m là tổng số máy (Machines).
- **Thân dữ liệu (Data Body):** Gồm đúng n dòng tiếp theo. Mỗi dòng tương ứng với một công việc (Job). Trên mỗi dòng sẽ có m cặp giá trị số nguyên $(M_{i,j}, T_{i,j})$ sắp xếp liên tiếp.
 - $M_{i,j}$: Chỉ số (ID) của máy thực hiện tác vụ thứ j của công việc i .
 - $T_{i,j}$: Thời gian xử lý của tác vụ đó trên máy $M_{i,j}$.

4.2.2. Ví dụ minh họa thực tế

Dưới đây là phân tích nội dung của file la01.txt (10×5):

10 5
1 21 0 53 4 95 3 55 2 34

0 21 3 52 4 16 2 26 1 71
...

- **Dòng đầu tiên (10 | 5):** Xác định bài toán có 10 công việc cần thực hiện trên 5 máy khác nhau.
- **Dòng thứ hai (Công việc 1 - Job 1):**
 - Tác vụ 1: Thực hiện trên Máy 1 trong 21 đơn vị thời gian.
 - Tác vụ 2: Thực hiện trên Máy 0 trong 53 đơn vị thời gian.
 - Tiếp tục cho đến tác vụ cuối cùng (thứ 5) trên Máy 4.
- **Các dòng còn lại:** Tương tự cho Job 2 đến Job 10.

4.2.3. Chuyển đổi dữ liệu vào chương trình

Trong quá trình lập trình, nhóm đã xây dựng module `data_loader` để đọc cấu trúc trên và lưu trữ vào hai ma trận chính:

1. **Ma trận thời gian xử lý (Processing Time Matrix - P):** Kích thước $n \times m$, lưu trữ các giá trị $T_{i,j}$.
2. **Ma trận thứ tự máy (Machine Sequence Matrix - M):** Kích thước $n \times m$, lưu trữ thứ tự các máy mà mỗi Job phải đi qua.

Việc tách biệt hai ma trận này giúp cho hàm tính Makespan và thuật toán SA có thể truy xuất dữ liệu với độ phức tạp thuật toán $O(1)$, từ đó tối ưu hóa tốc độ thực hiện khi chạy hàng nghìn vòng lặp với các bộ dữ liệu lớn như la36 - la40.

4.3. Lựa chọn kịch bản thực nghiệm

Để đánh giá toàn diện hiệu quả của giải thuật Simulated Annealing (SA) cải tiến, nhóm đã xây dựng kịch bản thực nghiệm tập trung vào việc thử thách thuật toán trên các quy mô dữ liệu khác nhau từ bộ Lawrence (LA).

4.3.1. Mục tiêu kịch bản

Kịch bản thực nghiệm được thiết kế nhằm trả lời ba câu hỏi lớn:

1. **Tính chính xác:** Thuật toán có thể tìm được nghiệm tối ưu (BKS) ở các bài toán quy mô nhỏ hay không?
2. **Tính ổn định:** Khi quy mô bài toán tăng lên (tăng số máy và số công việc), sai số của thuật toán so với BKS có nằm trong ngưỡng cho phép?
3. **Hiệu quả cải tiến:** Cơ chế Reheating và Adaptive Cooling có giúp thuật toán vượt qua các "bẫy" tối ưu cục bộ trong không gian nghiệm phức tạp của các bộ dữ liệu lớn?

4.3.2. Danh sách các bài toán thực nghiệm chọn lọc

Thay vì chạy dần trải cả 40 bộ, nhóm lựa chọn 08 bộ dữ liệu tiêu biểu đại diện cho 3 cấp độ khó khác nhau để phân tích sâu trong báo cáo:

Bảng 4.2. Danh sách các bài toán thực nghiệm

Nhóm kịch bản	Mã bộ dữ liệu	Kích thước (n×m)	Ý nghĩa thực nghiệm
Nhóm I (Dễ)	LA01, LA05	10×5	Kiểm tra tính đúng đắn và khả năng đạt BKS nhanh chóng.
Nhóm II (Trung bình)	LA16, LA20	10×10	Đánh giá độ ổn định khi tăng gấp đôi số lượng máy.
Nhóm III (Khó)	LA21, LA25	15×10	Kiểm chứng khả năng hội tụ trong không gian nghiệm lớn.
Nhóm IV (Cực khó)	LA36, LA40	15×15	Thử thách tối đa các cơ chế cải tiến (Reheating / Adaptive Cooling).

4.3.3. Quy trình thực hiện thực nghiệm

Do Simulated Annealing (SA) là thuật toán tối ưu hóa dựa trên xác suất, kết quả tìm kiếm có thể thay đổi giữa các lần chạy dù cùng một dữ liệu đầu vào. Vì vậy, nhóm thiết lập quy trình thực nghiệm chuẩn hóa để đảm bảo tính khách quan và khả năng tái lập kết quả.

4.3.3.1. Số lần chạy và tính tái lập

Mỗi kịch bản thực nghiệm (mỗi bộ dữ liệu LA) được thực hiện chạy 05 lần độc lập. Để đảm bảo tính minh bạch và khả năng kiểm chứng, nhóm sử dụng các giá trị hạt giống ngẫu nhiên (Random Seed) cố định cho từng lượt thử nghiệm (Trial):

- **Trial 1-5:** Tương ứng với *seed* {1001, 2002, 3003, 4004, 5005}. Việc quy định *seed* cho phép tái tạo chính xác các bước nhảy của thuật toán trong không gian nghiệm khi cần thực hiện đối soát.

4.3.3.2. Chỉ số ghi nhận và đánh giá

Kết quả thực nghiệm được tổng hợp dựa trên hai chỉ số quan trọng:

- **Makespan tốt nhất (C_{max}^{best}):** Giá trị nhỏ nhất tìm được trong 5 lần chạy, đại diện cho năng lực tối ưu cao nhất của thuật toán.
- **Makespan trung bình (C_{max}^{avg}):** Giá trị trung bình của 5 lần chạy, dùng để đánh giá độ ổn định (stability) của thuật toán.

Từ đó, nhóm tính toán độ lệch tương đối (Gap%) so với nghiệm chuẩn (BKS):

$$Gap_{best}(\%) = \frac{C_{max}^{best} - BKS}{BKS} \times 100$$

$$Gap_{avg}(\%) = \frac{C_{max}^{avg} - BKS}{BKS} \times 100$$

Ngoài ra, để đánh giá hiệu suất và chi phí tính toán của giải thuật, nhóm tiến hành đo lường thời gian thực thi:

- **Tổng thời gian thực thi (T_{total}):** Tổng thời gian máy tính cần để hoàn thành toàn bộ 05 lượt chạy (trial) độc lập cho một bộ dữ liệu.
- **Thời gian thực thi trung bình (T_{avg}):** Thời gian xử lý trung bình cho mỗi lượt chạy, đơn vị tính bằng giây (s).

$$T_{avg} = \frac{T_{total}}{5}$$

4.4. Thiết lập thông số và môi trường thực thi

Để đảm bảo tính chuẩn xác cho các kết quả thực nghiệm, nhóm tiến hành thiết lập môi trường phần mềm và tinh chỉnh các thông số vận hành cốt lõi của giải thuật Simulated Annealing.

4.4.1. Môi trường và dữ liệu

Toàn bộ quy trình thực nghiệm được xây dựng và triển khai trên nền tảng ngôn ngữ lập trình Python, cụ thể:

- **Ngôn ngữ:** Python 3.10+.
- **Thư viện hỗ trợ:**
 - **NumPy:** Tối ưu hóa tính toán ma trận thời gian xử lý (P, M) và các phép toán số học trong thuật toán SA.
 - **Matplotlib:** Trình diễn biểu đồ Gantt Chart (lịch làm việc máy) và đồ thị hội tụ Makespan (convergence curve).
 - **Pillow (PIL):** Xử lý và quản lý hình ảnh trong giao diện GUI, hỗ trợ tích hợp hình ảnh biểu đồ.
 - **Tkinter:** Xây dựng giao diện người dùng (GUI) đa tab cho phép người dùng tương tác trực tiếp với thuật toán.

- **Time:** Đo đạc thời gian thực thi từng trial độc lập, cung cấp chỉ số hiệu năng.
- **Random:** Kiểm soát hạt giống ngẫu nhiên (Seed: 1001, 2002, 3003, 4004, 5005) đảm bảo tái lập kết quả và độc lập thống kê giữa các trial.
- **Threading:** Quản lý luồng thực thi song song, cho phép giao diện GUI phản hồi mượt mà khi thuật toán chạy
- **Sys:** Quản lý đường dẫn dự án (sys.path) để import các module từ các thư mục khác nhau.
- **Dữ liệu thực nghiệm:** Các tập tin dữ liệu chuẩn la01.txt đến la40.txt được trích xuất trực tiếp từ bộ dữ liệu Lawrence thuộc thư viện OR-Library.

4.4.2. Thiết lập thông số thuật toán

Thông qua quá trình thực nghiệm sơ bộ, nhóm đã xác định được bộ thông số tối ưu giúp thuật toán đạt hiệu quả cao nhất về cả chất lượng nghiệm lẫn thời gian thực thi:

Bảng 4.3. Các thông số thuật toán

Thành phần	Ký hiệu	Giá trị	Ý nghĩa
Nhiệt độ khởi tạo	T_0	1000	Mức nhiệt độ ban đầu để bắt đầu quá trình thăm dò rộng
Nhiệt độ dừng	T_{min}	0.01	Ngưỡng kết thúc thuật toán, khi $T < T_{min}$ dừng tìm kiếm
Hệ số làm lạnh - Giai đoạn thăm dò	$\alpha_{explore}$	0.98	Tốc độ giảm nhiệt độ chậm ở giai đoạn đầu, khuyến khích khám phá
Hệ số làm lạnh - Giai đoạn khai thác	$\alpha_{exploit}$	0.95	Tốc độ giảm nhiệt độ nhanh ở giai đoạn sau, tập trung tinh chỉnh

Thành phần	Ký hiệu	Giá trị	Ý nghĩa
Markov chain length	L	150	Số lần lặp thử nghiệm ở mỗi mức nhiệt độ
Early stopping (Patience)	-	500	Dừng sớm nếu không có cải thiện trong 500 vòng lặp liên tiếp
Ngưỡng cải thiện	ε	1%	Phần trăm cải thiện tối thiểu để chuyển sang giai đoạn khai thác
Hệ số hâm nóng	f_{reheat}	0.85	Hệ số để tăng lại nhiệt độ khi Early Stopping kích hoạt ($T_{new} = T_{old} \div f_{reheat}$)
Xác suất SWAP	P_{swap}	0.5 (50%)	Xác suất chọn toán tử SWAP, 50% còn lại là MOVE
Số lần chạy độc lập	N_{trials}	5	Số lần thực thi thuật toán với các seed khác nhau
Hạt ngẫu nhiên (Seeds)	-	1001, 2002, 3003, 4004, 5005	Các seed cố định để đảm bảo tái lập kết quả

4.5. Kết quả thực nghiệm và phân tích trực quan

Phần này trình bày chi tiết kết quả thực nghiệm của thuật toán SA đã được tinh chỉnh trên bộ dữ liệu benchmark của Lawrence (1984). Các thí nghiệm được thực hiện trên 8 bài toán đại diện, được phân thành 4 nhóm dựa trên kích thước và độ khó: Dễ, Trung bình, Khó và Cực khó.

Để đảm bảo tính khách quan và ổn định của kết quả, mỗi bài toán được chạy độc lập 5 lần với các seed ngẫu nhiên khác nhau (1001, 2002, 3003, 4004, 5005). Các chỉ số được ghi nhận bao gồm:

- Best Gap (%): Độ lệch tương đối giữa makespan tốt nhất tìm được so với giá trị BKS.
- Avg Gap (%): Độ lệch tương đối trung bình của makespan sau 5 lần chạy so với giá trị BKS.

- Avg Time (s): Thời gian chạy trung bình của mỗi lần thực thi.

4.5.1. Tổng hợp kết quả thực nghiệm

Bảng dưới đây tổng hợp kết quả của 8 bài toán, được nhóm theo độ khó tăng dần:

Bảng 4.4. Tổng hợp kết quả thực nghiệm

Nhóm	Bài toán	Kích thước	Makespan	BKS	Best Gap (%)	Avg Gap (%)	Avg Time (s)
Dễ	LA01	10×5	666	666	0.00	0.00	0.870
	LA05	10×5	593	593	0.00	0.00	0.879
Trung bình	LA16	10×10	982	945	3.92	4.51	1.582
	LA20	10×10	907	902	0.55	1.69	2.559
Khó	LA21	15×10	1079	1046	3.15	5.03	2.236
	LA25	15×10	1032	977	5.63	7.76	2.231
Cực khó	LA36	15×15	1315	1268	3.71	5.58	3.514
	LA40	15×15	1278	1222	4.58	6.30	3.316

4.5.2. Phân tích chi tiết

Nhóm dễ (LA01, LA05): Với kích thước nhỏ (10x5), thuật toán SA tỏ ra cực kỳ hiệu quả, luôn tìm thấy lời giải tối ưu (makespan bằng với BKS) trong tất cả các lần chạy, thể hiện qua “Best Gap” và “Avg Gap” đều là 0.00%. Thời gian chạy trung bình rất nhanh, dưới 1 giây.

Nhóm trung bình (LA16, LA20): Kích thước bài toán tăng lên (10x10). Thuật toán vẫn cho kết quả rất tốt, đặc biệt là với “LA20” khi “Best Gap” chỉ là 0.55%. Kết quả của “LA16” (“Best Gap” 3.92%) cũng ở mức cạnh tranh. “Avg Gap” thấp cho thấy thuật toán hoạt động ổn định trên các bài toán có độ phức tạp vừa phải này.

Nhóm khó (LA21, LA25): Khi số lượng jobs tăng lên (15x10), không gian tìm kiếm trở nên lớn hơn đáng kể. “Best Gap” tìm được nằm trong khoảng

3.15% - 5.63%. Đáng chú ý, “Avg Gap” ở nhóm này (5.03% - 7.76%) cao hơn rõ rệt so với “Best Gap”, cho thấy thuật toán gặp nhiều khó khăn hơn trong việc tìm kiếm lời giải chất lượng cao một cách nhất quán. Sự khác biệt giữa các lần chạy lớn hơn, cho thấy một số lần chạy có thể bị mắc kẹt ở các điểm tối ưu cục bộ.

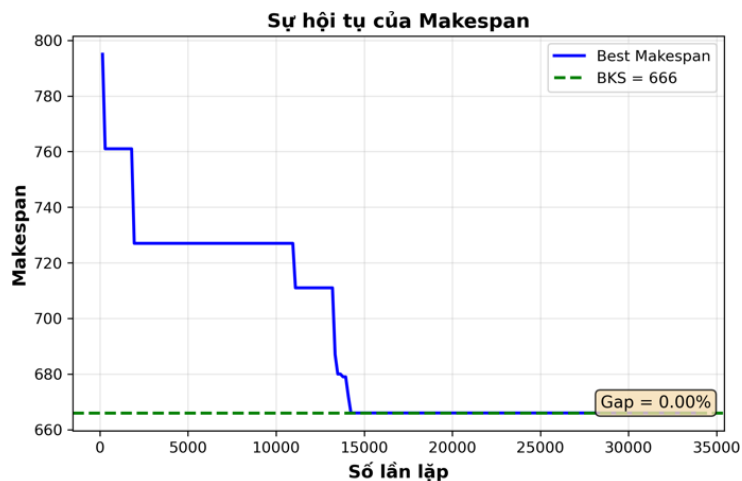
Nhóm cực khó (LA36, LA40): Đây là các bài toán có kích thước lớn nhất (15x15). “Best Gap” ở nhóm này (3.71% - 4.58%) vẫn giữ ở mức cạnh tranh và thậm chí còn tốt hơn so với “LA25”. Điều này có thể được giải thích bởi các tham số của SA (“ $L=150$ ”, “ $f_{reheat}=0.85$ ”) được điều chỉnh phù hợp hơn cho các không gian tìm kiếm lớn, cho phép thuật toán thoát khỏi các điểm tối ưu cục bộ hiệu quả hơn. Mặc dù vậy, “Avg Gap” vẫn còn tương đối cao (5.58% - 6.30%), khẳng định tính thách thức của các bài toán này.

4.5.3. Trực quan hóa và phân tích hành vi thuật toán

Việc phân tích các biểu đồ sẽ giúp hiểu rõ hơn về cách thuật toán hoạt động và lý do tại sao kết quả lại khác nhau giữa các nhóm.

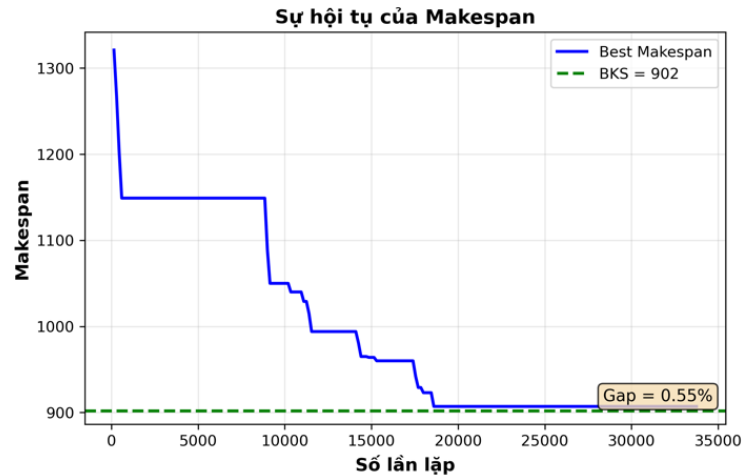
4.5.3.1. Phân tích biểu đồ hội tụ

Biểu đồ hội tụ ghi lại giá trị makespan tốt nhất tìm được qua từng vòng lặp, cho thấy "con đường" mà thuật toán đã đi để tìm kiếm lời giải.



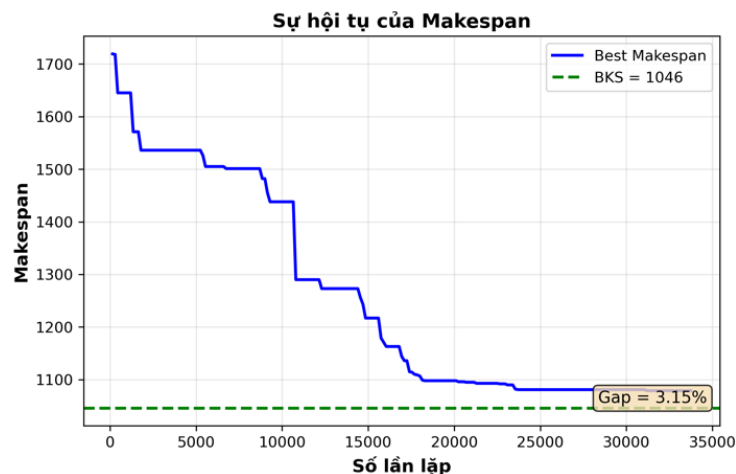
Hình 4.1. Biểu đồ hội tụ của LA01

Nhóm dễ (ví dụ LA01): Biểu đồ hội tụ có dạng dốc đứng ở giai đoạn đầu và nhanh chóng đạt đến một đường thẳng nằm ngang ở giá trị tối ưu. Điều này cho thấy thuật toán tìm ra lời giải tốt nhất rất nhanh và không có sự cải thiện nào thêm sau đó vì đã đạt đến cận dưới.



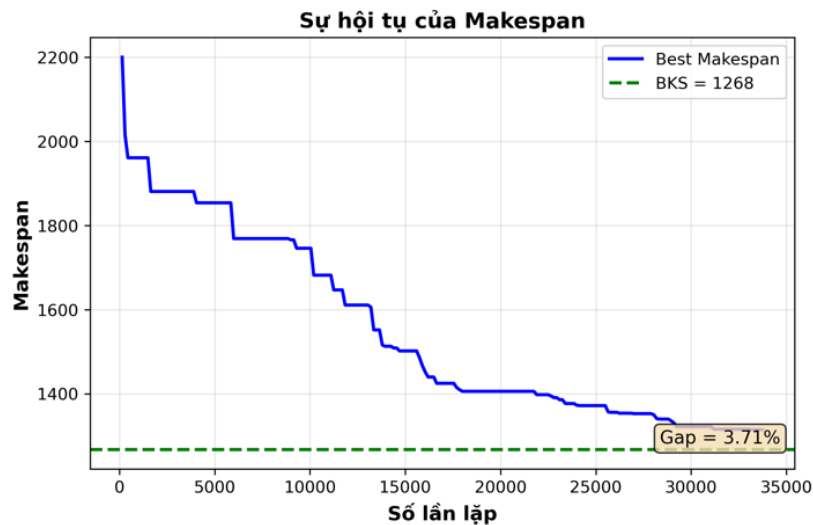
Hình 4.2. Biểu đồ hội tụ của LA20

Nhóm trung bình (ví dụ LA20): Quá trình hội tụ vẫn có dạng bậc thang nhưng các bước nhảy cải thiện lời giải diễn ra thường xuyên hơn so với nhóm dễ. Thuật toán dành nhiều thời gian hơn để "khai thác" ở các mức makespan khác nhau trước khi tìm được bước nhảy vọt tiếp theo. Kết quả “Best Gap” rất thấp (0.55%) cho thấy thuật toán vẫn hoạt động hiệu quả và tìm được lời giải rất gần với tối ưu.



Hình 4.3. Biểu đồ hội tụ của LA21

Nhóm khó (ví dụ LA21): Đồ thị hội tụ của nhóm này cho thấy rõ sự phức tạp tăng lên. Quá trình tìm kiếm trở nên "gập ghềnh" hơn với nhiều bước cải thiện nhỏ và liên tục, thay vì các bước nhảy lớn. Cơ chế hâm nóng (reheating) bắt đầu thể hiện vai trò rõ hơn qua các "gai" nhỏ, giúp thuật toán thoát khỏi các điểm tối ưu cục bộ. Sự chênh lệch giữa “Best Gap” và “Avg Gap” cũng được thể hiện qua sự phân tán của các đường hội tụ giữa các lần chạy.

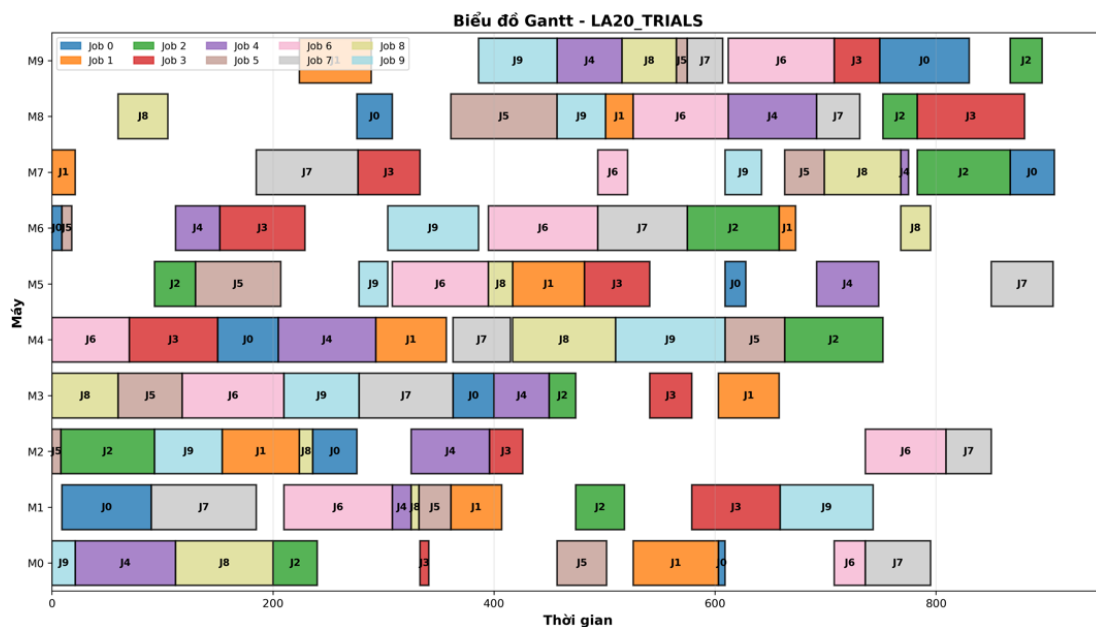


Hình 4.4. Biểu đồ hội tụ của LA36

Nhóm cực khó (ví dụ LA36): Đồ thị có dạng bậc thang dốc và rất gập ghềnh, bao gồm vô số các bước cải thiện nhỏ và liên tục. Điều này cho thấy một quá trình tìm kiếm rất chi tiết và vất vả trong một không gian lời giải phức tạp. Vai trò của cơ chế hâm nóng (reheating) được thể hiện một cách gián tiếp: sau các giai đoạn đi ngang ngắn (có nguy cơ bị kẹt), thuật toán vẫn liên tục tìm được các bước nhảy xuống để cải thiện lời giải, chứng tỏ khả năng thoát khỏi tối ưu cục bộ thành công. Đường cong hội tụ vẫn còn xu hướng đi xuống ở những vòng lặp cuối, gợi ý rằng nếu có thêm thời gian tính toán, kết quả có thể còn được cải thiện thêm.

4.5.3.2. Trực quan hóa lời giải

Trong khi biểu đồ hội tụ phân tích “quá trình”, biểu đồ Gantt minh họa cho “kết quả cuối cùng”. Nó trực quan hóa lịch trình sản xuất tối ưu mà thuật toán đã tìm ra.



Hình 4.5. Biểu đồ Gantt cho lời giải tốt nhất của LA20

Quan sát biểu đồ Gantt của bài toán “LA20”, ta có thể thấy một số đặc điểm của một lời giải tốt:

- Lịch trình được sắp xếp khá "đặc", các công việc được bố trí san sát nhau trên hầu hết các máy, cho thấy hiệu suất sử dụng tài nguyên cao.
- Mặc dù vẫn tồn tại các khoảng thời gian rỗi (idle time) do ràng buộc công nghệ, nhưng không có những khoảng trống quá lớn, chứng tỏ lịch trình đã được tối ưu hiệu quả.
- Điểm kết thúc của công việc cuối cùng trên biểu đồ xác định giá trị makespan tốt nhất (907) mà thuật toán tìm được.

⇒ Do đó, biểu đồ Gantt được dùng như một minh chứng trực quan cho chất lượng của lời giải mà thuật toán đã tạo ra.

4.6. Đánh giá hiệu quả của các cải tiến

Hiệu quả của thuật toán SA được trình bày trong các mục trên không chỉ đến từ bản chất của thuật toán gốc, mà còn từ các cải tiến quan trọng được tích hợp nhằm tăng cường khả năng tìm kiếm và thoát khỏi các điểm tối ưu cục bộ. Phần này sẽ đánh giá vai trò và hiệu quả của các cải tiến đó.

4.6.1. Cơ chế làm lạnh thích nghi

Thuật toán sử dụng một lịch trình làm lạnh hai giai đoạn với hai hệ số khác nhau: $\alpha_{explore}$ (0.98) cho giai đoạn đầu và $\alpha_{exploit}$ (0.95) cho giai đoạn sau.

Hiệu quả:

- **Giai đoạn khám phá (Exploration):** Với α cao (gần 1), nhiệt độ giảm chậm, cho phép thuật toán chấp nhận cả những lời giải kém hơn trong thời gian dài. Điều này thúc đẩy quá trình "khám phá" các vùng rộng lớn của không gian tìm kiếm, tránh việc hội tụ quá sớm vào một lời giải tốt nhưng chưa phải tối ưu. Điều này được thể hiện rõ trên các biểu đồ hội tụ ở giai đoạn đầu, khi makespan giảm rất nhanh.
- **Giai đoạn khai thác (Exploitation):** Khi nhiệt độ giảm xuống một ngưỡng nhất định, hệ số α thấp hơn được áp dụng, làm nhiệt độ giảm nhanh hơn. Thuật toán trở nên "tham lam" hơn, tập trung vào việc "khai thác" và tinh chỉnh xung quanh các khu vực có lời giải tốt đã tìm thấy.

Kết luận: Cơ chế này tạo ra sự cân bằng thông minh giữa việc tìm kiếm trên diện rộng và tinh chỉnh cục bộ, giúp thuật toán vừa có khả năng tìm ra các khu vực hứa hẹn, vừa có khả năng hội tụ nhanh khi đã ở gần điểm tối ưu.

4.6.2. Cơ chế chấp nhận nghiệm và kiểm soát sự hội tụ

Cơ chế này là sự kết hợp giữa tiêu chuẩn chấp nhận nghiệm Metropolis và kỹ thuật hâm nóng (reheating), đóng vai trò then chốt trong việc giúp thuật toán thoát khỏi các điểm tối ưu cục bộ.

Hiệu quả:

- **Thoát khỏi tối ưu cục bộ:** Vai trò của hâm nóng được thể hiện rõ nhất trên các biểu đồ hội tụ của các bài toán khó như “LA21” và “LA36”. Khi đồ thị đi ngang (dấu hiệu bị kẹt), một cú "sốc" nhiệt độ sẽ giúp thuật toán chấp nhận các nước đi xấu, từ đó "nhảy" ra khỏi thung lũng tối ưu cục bộ và tiếp tục tìm kiếm ở một khu vực khác.
- **Cải thiện kết quả ở bài toán khó:** Kết quả thực nghiệm cho thấy “Best Gap” của nhóm cực khó (LA36, LA40) lại tốt hơn một số bài toán ở nhóm khó (LA25). Điều này chứng tỏ cơ chế hâm nóng hoạt động đặc biệt hiệu quả trên các không gian tìm kiếm khổng lồ và phức tạp, nơi khả năng bị mắc kẹt là rất cao. Nếu không có cơ chế này, thuật toán có thể đã dừng lại ở một lời giải kém chất lượng hơn nhiều.

Kết luận: cơ chế này là "van an toàn" của thuật toán. Nó cho phép SA vừa duy trì khả năng khám phá cơ bản, vừa có một chiến lược mạnh mẽ để can thiệp khi quá trình tìm kiếm bị đình trệ, đảm bảo hiệu quả trên cả những bài toán khó nhất.

4.7. Tóm tắt chương

Chương 4 trình bày toàn diện quá trình thực nghiệm và đánh giá hiệu quả của thuật toán Simulated Annealing (SA) cải tiến trên bộ dữ liệu benchmark Lawrence (LA). Dựa trên một kịch bản chặt chẽ với 8 bài toán đại diện và 5 lần chạy độc lập cho mỗi bài, kết quả cho thấy thuật toán đạt hiệu suất cao, tìm được lời giải tối ưu cho các bài toán dễ và cho kết quả cạnh tranh trên các bài toán khó và cực khó. Thông qua phân tích biểu đồ hội tụ và Gantt, chương đã làm rõ vai trò then chốt của hai cải tiến cốt lõi: “Cơ chế làm lạnh thích nghi” giúp cân bằng giữa khám phá và khai thác, cùng “Cơ chế kiểm soát hội tụ và tái khám phá” đóng vai trò như một "van an toàn" giúp thuật toán thoát khỏi các điểm tối ưu cục bộ. Các bằng chứng thực nghiệm đã khẳng định rằng thuật toán SA được đề xuất là một phương pháp mạnh mẽ và hiệu quả để giải quyết bài toán lập lịch Job Shop.

KẾT LUẬN VÀ KHUYẾN NGHỊ

1. Kết luận

Đồ án đã hoàn thành tốt các mục tiêu đề ra ban đầu, đi sâu từ nghiên cứu lý thuyết đến xây dựng thành công chương trình giải quyết bài toán lập lịch Job Shop bằng thuật toán Simulated Annealing (SA) cải tiến. Thông qua toàn bộ quá trình thực hiện, nhóm nghiên cứu rút ra những kết luận quan trọng sau:

- **Về mặt lý thuyết - Thuật toán:** Đồ án đã hệ thống hóa mô hình toán học phức tạp của JSSP và chuyển đổi thành công sang không gian tìm kiếm của giải thuật SA thông qua kỹ thuật mã hóa Operation-based. Đặc biệt, nhóm đã tự phân tích và khắc phục điểm yếu của SA kinh điển bằng 2 cơ chế quan trọng: (1) ****Cơ chế làm lạnh thích nghi** giúp giải thuật giữ cân bằng chặt chẽ giữa việc khám phá (Exploration) và khai thác (Exploitation); (2) **Cơ chế chấp nhận nghiệm và kiểm soát sự hội tụ** được ứng dụng như một "van an toàn" mạnh mẽ, cho phép chương trình dễ dàng vượt qua các điểm cực trị địa phương (tối ưu cục bộ).
- **Về mặt thực tiễn - Kết quả:** Các kỹ thuật cải tiến đã được kiểm chứng trên bộ dữ liệu đa dạng Lawrence (LA) từ thư viện quốc tế OR-Library. Ở các bài toán quy mô nhỏ, thuật toán luôn đạt được cực trị (Gap = 0%). Đối với những bài toán khó và cực khó, SA cải tiến vẫn linh hoạt tìm được kết quả tiệm cận với Best Known Solution (BKS) với thời gian thực thi ổn định. Các đồ thị trực quan (Convergence Plot, Gantt Chart) đã phản ánh hiệu suất của giải thuật, đáp ứng tính thực tiễn cao cho việc điều hành sản xuất.

2. Khuyến nghị

Mặc dù giải thuật SA đạt kết quả rất tiềm năng, tuy nhiên quá trình xếp lịch phân xưởng tại các doanh nghiệp thực tế luôn tiềm ẩn nhiều biến số phát sinh. Từ kết quả trên, nhóm đề xuất một số hướng phát triển tiếp theo của đề tài:

- **Mở rộng thành bài toán JSSP động (Dynamic JSSP):** Đưa thêm các tham số nhiễu từ môi trường sản xuất thực tế như thiết bị hỏng đột xuất, có đơn đặt hàng ưu tiên chen ngang, hoặc thời gian vận chuyển bị kéo dài. SA sẽ cần kết hợp thêm các bộ quy tắc (Priority Rules) linh hoạt để sửa lịch theo thời gian thực.
- **Lai ghép giải thuật (Hybrid Metaheuristics):** Áp dụng sự giao thoa giải thuật. Có thể sử dụng SA làm cơ chế tìm kiếm cục bộ tinh chỉnh, trong khi dùng một giải thuật tiến hóa (như Genetic Algorithm) hoặc bầy đàn (PSO, ACO) để chạy phân tán tìm kiếm toàn cục, tối ưu chất lượng lời giải.
- **Nâng cấp ứng dụng triển khai:** Có thể đưa mô hình lập trình Python đóng gói thành lõi API Web Service để các nhà máy có hệ thống MES (Manufacturing Execution System) hay ERP (Enterprise Resource Planning) dễ dàng gọi dữ liệu và truy xuất biểu đồ Gantt phân công công việc qua nền tảng Cloud trực tuyến.

TÀI LIỆU THAM KHẢO

- [1]. **Phan Quốc Khánh**, “Vận Trù học”, NXB Giáo dục, 2006.
- [2]. **Nguyễn Đức Nghĩa**, “Tối ưu hóa: Quy hoạch tuyến tính và rời rạc”, NXB Giáo dục, 1996.
- [3]. **Nguyễn Hữu Mùi**, “Thuật toán và các bài toán lịch biểu”, Luận án tiến sĩ công nghệ thông tin, Trường Đại học Công nghệ, Đại học quốc gia Hà Nội, 2013.
- [4]. **Lê Minh Tuấn**, “Nghiên cứu đề xuất cải tiến thuật toán lập lịch và ứng dụng”, Luận án tiến sĩ công nghệ thông tin, Viện công nghệ thông tin, Đại học quốc gia Hà Nội, 2022.
- [5]. **Phạm Thế Quế**, “Các thuật toán tối ưu - Lý thuyết và ứng dụng”, NXB Bru điện, 2008.
- [6]. **Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P.**, “Optimization by Simulated Annealing”, Science, 220(4598), 1983.
- [7]. **Pinedo, M. L.**, Scheduling: Theory, Algorithms, and Systems, 5th Edition, Springer, 2016.
- [8]. **Talbi, E. G.**, Metaheuristics: From Design to Implementation, John Wiley & Sons, 2009.
- [9]. **Van Laarhoven, P. J., & Aarts, E. H.**, Simulated Annealing: Theory and Applications, Springer Science & Business Media, 1987.
- [10]. **Đỗ Ngọc Nhung**, Nghiên cứu và xây dựng thuật toán Bees giải bài toán Job Shop Scheduling, Luận văn thạc sĩ Khoa học máy tính, Trường Đại học Sài Gòn, 2025.